# Precise Behavioral Code Synthesis in Leant and Djex with Z3

A concrete architecture for synthesis from
type signatures plus behavioral contracts

Counterexample-guided search, typed sketch completion, semantic pruning, optimization, recursive verification, and Lean-checked proof artifacts

Technical proposal for the Leant and Djex projects

August 15, 2026

Based on the exact repository revisions listed in section 1.3.

# Abstract

LEANT and DJEX already solve the type-directed half of program synthesis unusually well: they combine terminating logical proof search, ranked heuristic search, exact source-to-engine provenance, typed candidate graphs, provider instantiation evidence, and mandatory re-elaboration by the Lean backend. They also already contain a serious first behavioral layer. The current finite-spine *Length* domain can associate a verified candidate with an exact inventory and contract, symbolically interpret a checked candidate graph, lower the resulting counterexample problem to bounded canonical QF_LIA, run a capability-probed Z3 worker, and accept model input only after independent replay. In LEANT today this layer is deliberately a post-verification ranking stage: it may reorder candidates but does not prune them, and raw solver status carries no authority.

This proposal starts from that implemented state rather than from the older idea of merely "adding Z3." It describes a next-generation synthesis architecture in which behavioral information participates progressively earlier in the search while preserving the present trust boundaries. The core recommendation is a three-way division of responsibility:

- DJEX owns the typed term grammar, source identities, provider evidence, search fairness, and construction of exact candidates.
- Z3 owns finite discrete choice, model generation, arithmetic and data-theory reasoning, unsatisfiable cores, optimization, and constrained Horn-clause exploration.
- Lean 4 owns source elaboration, universe and type-class reconstruction, termination checking, and any final proof artifact that is advertised as established behavior.

The proposed system is staged. The first increment turns the existing Length machinery into an opt-in hard behavioral filter and a counterexample-guided inductive synthesis loop without changing the term grammar. The next increment lets Z3 solve finite *typed sketches*: Djex constructs a bounded, type-correct choice graph; Z3 selects productions that satisfy accumulated examples; Djex materializes a candidate; Lean re-elaborates it; and the behavioral domain searches for a fresh counterexample. Later increments add sound prefix pruning through over-approximate summaries, domain families for sequences, bags, constructor behavior, bit-precise arithmetic, state transitions and costs, and a constrained-Horn-clause path for recursive programs. Positive claims remain stratified: finite exhaustive replay can establish a bounded fact; a replayed model can refute a modeled contract; a Z3 `unsat` answer is a solver observation unless converted into a Lean-checkable obligation; and assumptions about provider behavior remain explicit hypotheses.

The result is not a monolithic translation of Lean into SMT. It is an extensible family of narrow, versioned behavioral abstractions, each with a checked source language, an exactness statement, a symbolic evaluator, a Z3 lowering, a replay procedure, and a Lean proof-obligation generator. This permits useful precision without pretending to solve arbitrary higher-order dependent program synthesis in one encoding.

# Executive summary

**Recommendation**

Implement the next vertical slice as **counterexample-guided hard filtering over the existing Length domain**, followed immediately by a **typed-sketch CEGIS lane**. Do not begin by generalizing every contract or by embedding the entire Lean calculus in Z3. The current exact-origin, typed-candidate, session, query, and replay machinery is already enough to prove the control architecture on a useful domain.

The proposal has eight principal conclusions.

1. **The current system already has the hard part of a trustworthy solver boundary.** DJEX's generic `BehavioralProblem` associates observations with inventory, encoding, candidate, and problem fingerprints; its public observation layer distinguishes reports from evidence; the Length domain independently replays model inputs; and LEANT binds that authority to the exact callback-accepted candidate occurrence. This should be generalized carefully, not replaced.

2. **Counterexamples should enter search before positive solver claims do.** A replayed counterexample is directly actionable: it can reject or demote a candidate, populate a persistent sample bank, split semantically equivalent candidates, and guide sketch construction. By contrast, a raw `unsat` answer should not silently become a proof. The first new mode should therefore use Z3 primarily as a counterexample generator.

3. **Typed sketch completion is the highest-leverage generative use of Z3.** Djex should enumerate a bounded type-correct grammar and expose finite selector variables, rather than asking Z3 to invent raw Lean syntax. Z3 then solves a compact finite constraint problem over production choices and concrete observations. This preserves the engines' existing knowledge of binders, rank-N instantiation, providers, constructors, and rendering.

4. **Semantic prefix pruning is valuable only with an explicit approximation direction.** A partial candidate may be pruned from an unsatisfiable completion query only when the symbolic summary is a proved over-approximation of all permitted completions. An under-approximation can find witnesses but cannot justify pruning. The API should encode this polarity nominally.

5. **Behavior should be factored into narrow domains, not one universal IR.** Length/shape, sequence indexing, bag equality, constructor cases, scalar arithmetic, bit-vectors, state relations, and cost recurrences have different exactness and proof stories. They can share fingerprints, process ownership, SMT syntax, response parsing, assumption tracking, and CEGIS orchestration without sharing one unsound "semantic value" type.

6. **Proof-producing integration with Lean is the route to positive authority.** For linear arithmetic, emit a Lean theorem and discharge it with `omega`; for finite propositions use `decide`; for bit-vectors use `bv_decide`; for recursive structures generate induction lemmas and invariants. Z3 proposes counterexamples, invariants, clause subsets, and candidate choices; Lean checks the final artifact.

7. **Z3 Optimize, unsat cores, and Spacer are secondary engines, not replacements for ordinary SAT**

**checks.** Optimize is useful for choosing among sketches after hard behavioral constraints; unsat cores can learn grammar nogoods and explain inconsistent contracts; Spacer can search for inductive invariants of recursive relational semantics. Each requires its own versioned protocol and status taxonomy.

8. **The implementation should remain additive.** The current Length contract and configuration schemas, startup behavior, and ranking output need not change: when no behavior mode is selected, the existing scalar and pair paths run exactly as they do today. A new generic behavioral layer can wrap the current Length implementation, while new hard-filter and CEGIS commands are opt-in and separately fingerprinted.

Readers who know Z3 but not this codebase should read section 1.2 first; the rest of the document uses the project's vocabulary (providers, occurrences, certificates, sealing, replay) without further definition. Readers who know the codebase can skip to chapter 3.

**Recommended order of construction:**

1. hard Length filtering with exact replay and conservative unknown handling;
2. query-scoped counterexample banks and ordinary candidate CEGIS;
3. a generic contract/session/problem envelope which adapts existing Length unchanged;
4. typed selector-DAG sketches for nonrecursive terms;
5. sequence and constructor-behavior domains;
6. Lean proof obligations for positive claims;
7. bag/permutation and bit-vector domains;
8. sound abstract-completion pruning;
9. CHC/Spacer support for recursive programs and invariants.

# Contents

# List of Figures

# List of Tables

Chapter 1

# Scope, inspected revisions, and current baseline

## 1.1 Scope of this proposal

The requested problem is synthesis from a type signature plus a behavioral constraint. The type gives a structural search space: which arguments exist, which constructors and functions may be composed, where binders occur, and which terms Lean can elaborate. The behavioral constraint distinguishes inhabitants that the type deliberately leaves indistinguishable. Typical examples are:

- a function of type `List a -> List a` that preserves length, preserves the multiset of elements, is exactly reverse, or is exactly identity;
- a state-monad bind which passes the state produced by the first computation to the second, rather than accidentally reusing the initial state;
- an `Option.bind` implementation which propagates `none` and calls the continuation exactly in the `some` case;
- a pair of lists which forms a stable partition of the input;
- an integer or machine-word function which satisfies a range, monotonicity, overflow, or bit-level law;
- a recursive tree transformation which preserves an inorder sequence while satisfying a balance or size invariant.

The proposal covers architecture, semantic contracts, Z3 encodings, algorithms, trust, proof artifacts, APIs, diagnostics, performance controls, testing, and an implementation roadmap. It does not propose to translate arbitrary Lean dependent types, effects, quotient types, proof irrelevance, or nontermination wholesale into SMT. Those features remain at the Lean boundary unless a particular behavioral domain supplies a precise model.

## 1.2 Terminology

This document uses LEANT's and DJEX's own vocabulary throughout. table 1.1 fixes the meaning of the terms that recur most often; each entry says what the word means *here*, which is sometimes narrower than its general use.

Table 1.1: Project vocabulary used in this proposal.

| Term | Meaning in this document |
|---|---|
| Provider | A Lean declaration (function, constructor, projection, instance-derived operation) that the synthesis inventory makes available for building candidates. The *provider inventory* is the goal-relevant subset retained for one target. |

| Term | Meaning in this document |
|---|---|
| Inventory | The checked collection of type families, constructors, providers and bindings one synthesis request may use, sealed under a fingerprint. Every later artifact names the inventory it was checked against. |
| Occurrence | One particular use of a provider or constructor inside a candidate graph, together with its exact type arguments. Two uses of the same provider are two occurrences; authority is granted per occurrence, never per name. |
| Certificate | DJEX evidence attached to a provider occurrence recording how its type arguments, constraints and instances were resolved. "Occurrence-specific certificate" means the certificate of that one use. |
| Ground discharge | Establishing that a provider's class constraints hold at the exact ground types of an occurrence. Some provider laws are admitted only *conditional on* such a discharge. |
| Provider law / transfer | An explicit, user-supplied assumption about how a provider transforms an observation (for example, "`List.reverse` preserves length"). Never inferred from names, source text or tests; classified as assumed uniformly, conditional on ground discharge, or Lean-proved. |
| Spine, spine family | A unary recursive datatype such as `List` with one *zero* constructor (`List.nil`) and one *step* constructor (`List.cons`). The Length domain observes only the spine, that is, the length; elements are opaque. |
| Role, role vector | The source-ordered assignment of one modeling role to each target argument (observed spine, opaque value, callback, . . . ). Roles are declared explicitly in the contract and arity-checked; they are never inferred from a type constructor name. |
| Callback (verification) | LEANT's re-elaboration of each candidate spelling by the Lean backend against the exact requested type. The first spelling the backend accepts yields the `Verified` receipt. |
| Callback (argument) | A function-typed input of the target, such as `f` in `(a -> Option b)`. Behavioral domains model such arguments relationally (as uninterpreted functions or call events); they never evaluate them. Context makes clear which sense is meant. |
| `Verified` receipt | An opaque value proving that the verification callback accepted one specific candidate occurrence. It is the only admission ticket to post-verification stages and is never upgraded in place into behavioral evidence. |
| Exact typed origin | The `ExactTypedVariantOrigin` sidecar tying an accepted spelling back to the typed candidate graph, the checked inventory, the source and search goals and the provider bindings that produced it. Behavioral domains consume this, not rendered text. |
| Sealing, sealed | Constructing an opaque checked value through its single authorized constructor after every check has passed. Session, contract and candidate problem are sealed in that order; nothing downstream can forge or reassociate them. |
| Handoff | `Leant.Synth.Length.Handoff`: the sole conversion of one `Verified` receipt plus one passive contract into a checked DJEX Length problem. It refuses mismatches instead of trusting caller-supplied graphs or names. |
| Fingerprint | The canonical, collision-resistant identity of a checked artifact (inventory, encoding, candidate, contract, problem, query). Evidence is bound to the tuple of fingerprints named in chapter 2, never to a rendered term or a hash of one. |
| Post-verification boundary | `Leant.Synth.PostVerification`: the stage after callback verification. Today it admits only a sealed *permutation* of the verified receipts, which is why ranking can reorder but never remove candidates. |

| Term | Meaning in this document |
|---|---|
| Replay | Independent re-evaluation of a decoded solver model by the domain's own evaluator against the exact sealed problem. A model becomes a counterexample only after replay reproduces the contract violation; raw solver output never does. |
| Applicable domain | The set of inputs satisfying a contract's precondition. Applicable-domain validation enumerates a bounded part of it exhaustively; its receipts record how many assignments were applicable, so a vacuous contract is distinguishable from a satisfied one. |
| Monus | Truncated natural subtraction, $n \dot{-} m = \max(n - m, 0)$; the arithmetic of a `List.cons` step case read backwards. |
| CEGIS | Counterexample-guided inductive synthesis: alternately propose a candidate consistent with all known samples and search for a new counterexample; stop when none is found or a budget ends. |
| Sketch | A partial program with holes. Here always a Djinn-prepared, type-correct DAG of alternatives; Z3 chooses among the alternatives, it never invents syntax. |
| Nogood | A learned clause forbidding a combination of choices that has been shown infeasible, scoped to the exact grammar and sample bank under which it was learned. |
| CHC, Spacer | Constrained Horn clauses, and Z3's fixedpoint engine for them, which returns either a derivation of the "bad" state or an inductive invariant. |
| Over-/under-approximation | A summary of a set of behaviors that contains at least (respectively at most) every real behavior. Only an over-approximation can justify pruning; an under-approximation can only produce witnesses. |

## 1.3 Exact repository revisions

The analysis is tied to the following repository heads observed on August 15, 2026. Full hashes are used because all three repositories are under active development.

The repositories are cited by immutable commit URLs in the bibliography. Statements about current module ownership are based primarily on Leant's `README.md`, `docs/synth-internals.md`, and `src/Leant/Synth/Length/*`, and on Djinn's `synthesis/README.md` and `docs/semantic-foundations.md`. Z3 capabilities are referenced to the official online guide and repository.

**Changes since the pinned revisions.** Later on the same day Leant replaced its numbered Length schemas with versionless ones: the startup configuration in commit `4fa5c0a1` and the command-local contract file in `d4104c91`, documented in `dd87538a` and `docs/reports/2026-08-15-versionless-length-contract.md`. The current root is one `format` literal, one `rankingDomain` discriminator (`scalar` or `binary-product`) and one `contract` object, with no `version` member, no migration decoder and an explicit statement that the experimental entrance carries no compatibility commitment. Neither change touches Djinn's checked semantics, the solver protocol, replay or evidence authority, so the architectural claims below stand unchanged. Where this proposal mentions "existing file versions" the phrase should be read as "the current versionless schemas", and its compatibility statements as "the current schemas need not change", not as a promise to freeze them; section 4.2 discusses how the versioned envelope proposed here relates to that policy. Djinn advanced from `0501e72e` to `43ec4f70` through refactoring and documentation moves only.

**Implemented since the pinned revisions (19 August 2026).** The first increment recommended in this proposal has since shipped: `:synth --behavior-mode filter` (section 6.1) performs command-authorized hard Length

Table 1.2: Repository revisions inspected for this proposal.

| Repository | Commit | Commit time (UTC) | Relevant state |
|---|---|---|---|
| Leant | `b701ee59360d6b04456119dc68395ab35326a8ee` | 2026-08-15 23:24 | Current `:synth`, exact semantic origins, verified-candidate receipts, and Length integration. |
| Djex | `0501e72edf3fb59f6800072b9ae105d4fdcb972b` | 2026-08-15 23:28 | Shared synthesis foundation, typed candidate graphs and certificates, generic behavioral problem envelope, Length domain, and live Z3 stack. |
| Z3 | `221b77f086b41fac32a761f20e26894f5be1518a` | 2026-08-15 21:48 | Current solver source. The latest stable release visible during research was 5.0.0, released July 17, 2026. |

filtering over verified candidates, in which only an independently replayed counterexample rejects a candidate, every other assessment (including `unknown` and heuristic `sat`/`unsat`) conservatively retains it, and rejection rows are reported rather than silently dropped. Command-local counterexample banks retain replayed input vectors for reuse inside one command, and the filter lane refills progressively within a single run; the dated reports under `docs/reports/` in both repositories record the exact boundaries. The typed-sketch CEGIS lane, sound prefix pruning, the wider domain families, and proof-producing positive verification remain unimplemented.

**What was executed.** The Lean listings in chapters 8 and 10 whose definitions appear in the text (the reverse, one-layer case, state-bind, `Option.bind` and provider-hypothesis theorems, and the accumulator-invariant proof shape) were type-checked with Lean 4.33.0. The six SMT-LIB scripts of chapter B were executed with Z3 5.0.0 and the observed answers are quoted after each listing. The Haskell interfaces are conceptual and were not compiled.

## 1.4 What Leant and Djex already do

Leant's `:synth TYPE` command is not a thin wrapper over a single proof search routine. At the inspected revision it combines two Djex engines and a substantial Lean-specific preparation and verification pipeline [2, 4, 11]:

- Djinn contributes terminating LJT-based proof search for an intuitionistic structural fragment and may produce a genuine uninhabitation result when translation is complete.
- Exference contributes ranked, bounded expression search over structural operations and a goal-relevant provider inventory.
- The preparation layer retains nominal Lean families, constructor schemas, provider bindings, proper-type applications, rank-N structure, active-instance-based provider assignments, renderer metadata, and explicit

completeness facts.

- Candidate groups may contain several Lean spellings of one semantic candidate. A callback tests variants in order, and the first accepted variant becomes an opaque `Verified` receipt.
- Every displayed candidate is re-elaborated by the Lean backend against the exact requested type. An engine bug therefore drops a candidate rather than admitting an ill-typed term.
- Negative results are labeled by strength. A complete lossless logical refutation is distinct from a bounded search miss.

This baseline matters because behavior should not be bolted onto rendered text. The current `DetailedVerification Variant` can retain an `ExactTypedVariantOrigin`: an opaque sidecar tying the accepted spelling back to the exact typed candidate, the checked inventory, source and search goals, provider bindings, renderer ordinal, and semantic-family provenance. The Length handoff consumes the *verified receipt*, recovers this origin internally, and refuses mismatches instead of trusting a caller-supplied graph or name.

## 1.5 The implemented Length behavioral domain

The current Length layer is already a deep implementation of the trust pattern needed by future domains [7, 8, 9, 12]. Its important properties are summarized below.

### 1.5.1 Passive, explicit contracts

LeanT's `LeanLengthContract` supplies an exact spine family and constructor identity, a source-ordered role vector, an explicit candidate case policy, a solver-neutral Djinn `LengthContractSource`, and explicitly assumed provider laws. Names are resolved against retained translation provenance. Provider transfer laws are never inferred from names or implementations.

### 1.5.2 Occurrence-sealed handoff

`prepareCheckedLengthProblem` accepts a passive contract and one callback-accepted `Verified DetailedVerificationVariant`, and turns them into a Djinn problem through a fixed sequence of fail-closed checks, each with its own refusal constructor:

1. the accepted variant must have come through the typed rendering route and carry the exact semantic sidecar (a Djinn text-only variant is refused, not guessed at);
2. the source and search fragments recorded in the sidecar must still be the ones the command targeted, and the request must have the plain shape the domain models: no caller or constructor premises and no request contexts, because a spine length observed under extra hypotheses would mean something else;
3. the original variant is rerendered and must reproduce the accepted text at the same renderer ordinal, so the graph being interpreted is the graph Lean accepted;
4. the contract's spine family, its constructors and every named provider are resolved against the retained provenance of that exact inventory (an unavailable or ambiguous name is a refusal, never a best match);
5. a Djinn session is sealed with one interpretation policy, the contract is sealed inside that session, and finally the whole typed candidate is sealed into a candidate-specific problem.

The product-domain analogue additionally requires a canonical Lean `Prod` result whose two fields are the configured spine.

### 1.5.3 Typed symbolic interpretation

The Djex Length interpreter handles a deliberately narrow term-graph fragment: locals, lambdas, applications, explicit type applications, tuples, lets and supported patterns, modeled spine constructors, exact zero/step cases under explicit authority, and provider transfers under checked occurrence-specific certificates. Observed list arguments become symbolic natural lengths. Unobserved arguments become opaque tokens which may be forwarded but not inspected or called. Unsupported demand fails closed.

For an exact spine case with symbolic length $n$, the semantics is essentially

$$\mathrm{caseLen}(n, z, s) = \mathrm{ite}(n = 0, z, s(n \dot{-} 1)),$$

where $\dot{-}$ is natural monus. Both branches are interpreted, and provider-law authority is the union of the branches even when normalization removes the conditional.

### 1.5.4 Canonical query, live solver, and replay

A checked problem is lowered to a bounded canonical QF_LIA counterexample query. The live stack owns executable admission, process identity, capability probing, reset/check/value transactions, causal framing, bounded parsing, cancellation, deadlines, and a shared query lease. A raw `sat`, `unsat`, or `unknown` reply is associated with exact problem fingerprints but remains heuristic. Model values are accepted only after the exact sealed problem independently evaluates them and reproduces a contract violation.

### 1.5.5 Ranking, not pruning

In Leant the current integration occurs after Lean callback verification. `Leant.Synth.PostVerification` permits only a sealed permutation of the exact verified receipts. Length assessment may replay a small MRU bank of counterexamples, perform bounded solver-independent domain validation, or run Z3, and then stably reorder candidates. It never omits a candidate. If assessment fails, callback order is preserved.

This conservative behavior is appropriate for a first semantic layer, but it also identifies the next architectural step: introduce a separately authorized *selection* boundary for opt-in hard constraints, rather than overloading the permutation-only ranking boundary.

## 1.6 The actual gap

The gap is no longer "Can Z3 say anything about a synthesized term?" It can, and the Length implementation already demonstrates a high-integrity answer. The remaining gap is that term generation and behavioral reasoning are mostly sequential:

$$\text{type-directed enumeration} \longrightarrow \text{Lean verification} \longrightarrow \text{behavioral ranking}.$$

Precise behavioral synthesis requires feedback loops:

$$\text{grammar/search} \rightleftarrows \text{examples/counterexamples} \rightleftarrows \text{solver constraints} \longrightarrow \text{Lean-checked candidate and proof}.$$

There are four increasingly ambitious ways to create those loops:

1. reject verified candidates contradicted by a checked behavioral counterexample;
2. retain counterexamples across candidates and prioritize or suppress candidates by their semantic signatures;

3. let Z3 choose productions in a bounded, type-correct sketch prepared by Djex;
4. use sound semantic summaries to prune incomplete search states, and use CHCs to reason about recursive relational semantics.

The rest of this document develops these levels while preserving the exact-origin and evidence boundaries already present.

Chapter 2

# Goals, non-goals, and governing invariants

## 2.1 Primary goals

The system should satisfy the following goals.

**Behavioral precision.**
A user can distinguish inhabitants of the same type by explicit preconditions, postconditions, examples, structural observations, relational laws, and optimization preferences.

**No regression in type soundness.**
Every emitted Lean term still passes the existing exact-goal elaboration callback. Behavioral integration cannot bypass, replace, or weaken that gate.

**Explicit semantic scope.**
Every contract identifies a domain, model, interpretation policy, provider assumptions, and exactness class. Unsupported source constructs produce a refusal, not an invented semantics.

**Counterexample productivity.**
A solver model becomes useful only after bounded decoding and independent replay. Once replayed, it can be reused as a test, a CEGIS sample, a diagnostic, and a search partition.

**Positive-proof path.**
A user can request a Lean theorem witnessing the behavioral contract. Z3 may synthesize an invariant or arithmetic certificate input, but the Lean kernel remains the final authority for claims presented as proved.

**Search compositionality.**
Djinn, Exference, provider discovery, typed sketches, and future engines can all consume the same contract and counterexample state without guessing one another's private syntax.

**Bounded operation.**
Candidate work, symbolic interpretation, query construction, solver IO, model decoding, replay, sample storage, sketch width, CHC exploration, and proof generation each have explicit independent limits.

**Reproducibility.**
Solver version, launch profile, parameters, encoding version, grammar fingerprint, objective order, and all semantic inputs enter the relevant execution and cache identities.

## 2.2 Non-goals

The first several phases should explicitly avoid the following goals.

- A complete translation of Lean's dependent type theory into first-order logic.
- Trusting an arbitrary Z3 model as an executable Lean value.
- Treating Z3 `unsat` as a Lean proof without an explicit trust mode or proof reconstruction.
- Inferring provider behavior by symbol name, documentation, source text, or testing alone.
- Solving unbounded higher-order extensional equality in general.
- Using one universal semantic IR for lists, state, bit-vectors, costs, recursion, and effects.
- Preserving every current textual configuration shape while simultaneously declaring a stable public contract language. LEANT's current Length schemas are experimental and carry no compatibility commitment (section 1.3); the proposal does not ask to freeze them. New formats should likewise remain experimental until their semantics settles.

## 2.3 Governing invariants

**Design rule**

A behavioral statement must always be attached to the exact tuple

$$(\text{domain}, \text{inventory}, \text{encoding}, \text{candidate}, \text{contract}, \text{policy}).$$

No cache key, diagnostic name, rendered term, graph hash, model, proof, or solver session may stand in for that tuple unless the domain has sealed the substitution.

**Design rule**

Search guidance and authority are separate. Semantic fingerprints, sample signatures, solver scores, learned clauses, and unsat cores may guide enumeration without becoming evidence. Evidence requires the domain's independent validation path; a proved claim additionally requires the configured proof authority.

**Fail-closed requirement**

Any translation that drops a source distinction relevant to the selected domain must degrade to *inapplicable* or *unknown*. It must not silently strengthen a precondition, weaken a postcondition, replace total functions by partial ones, equate distinct nominal families, erase overflow, or assume termination.

**Design rule**

All pruning rules state their approximation polarity. An over-approximation of completions may justify pruning when even the over-approximation is unsatisfiable. An under-approximation may produce witnesses but may never justify pruning. Exact encodings may do either.

## 2.4 A lattice of behavioral outcomes

A binary pass/fail result is insufficient. The proposed common status vocabulary should preserve at least the distinctions in table 2.1.

Table 2.1: Behavioral outcome classes and permitted uses.

| Outcome | Typical producer | Permitted consequence |
|---|---|---|
| Inapplicable | Contract/domain sealer | Candidate remains in ordinary type-directed search; report exact refusal. |
| Preparation refused | Typed-origin, inventory, or graph check | Candidate cannot receive this domain's authority; ordinarily retain candidate unless the user required the contract. |
| Counterexample observed | Raw Z3 model | No authority yet; decode and replay. |
| Counterexample replayed | Domain evaluator | Refute the modeled contract for the exact candidate, subject to the recorded provider assumptions and model scope; may reject in hard mode. |
| Validated within bounds | Exhaustive finite evaluator | Positive bounded receipt; may rank or satisfy an explicitly bounded contract, but does not assert an unbounded theorem. |
| Solver unsatisfiable | Z3 | Heuristic positive signal or input to proof reconstruction; never silently a Lean proof. |
| Behavior established by domain | Domain-specific complete checker | Authoritative only for the exact domain semantics and exactness statement; may still be conditional on provider laws. |
| Lean theorem accepted | Lean elaborator/kernel | Proof-grade behavioral result for the generated proposition in the active environment. |
| Unknown, timeout, or resource limit | Any stage | Preserve candidate by default; hard modes may treat it according to an explicit policy, never as success or failure implicitly. |
| Internal inconsistency | Fingerprint, replay, protocol, or association check | Fail the behavioral stage closed, invalidate the affected cache/session, and preserve the original synthesis result unless policy says the contract is mandatory. |

Chapter 3

# Proposed architecture

## 3.1 Three owners, one loop

The proposal assigns responsibilities according to the information each component can represent faithfully.

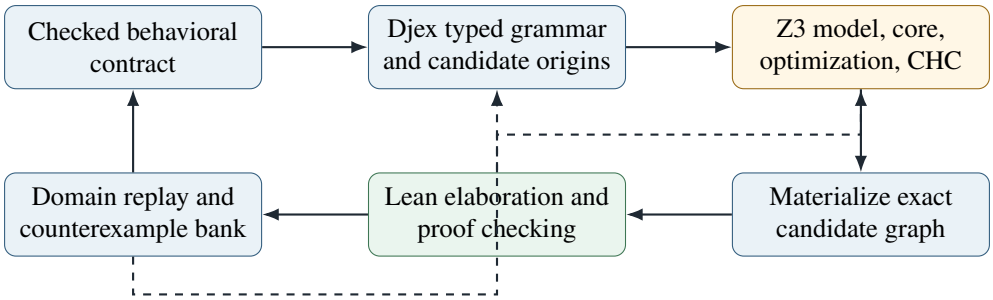


Figure 3.1: The proposed synthesis loop and ownership boundaries.

Djex should never ask Z3 to return an untyped string. It should give Z3 either (a) a counterexample formula for an exact candidate, (b) finite selector variables over a type-correct grammar, (c) a sound summary of all completions of a partial state, or (d) a CHC system whose relational meaning is separately defined. Z3 returns a model, core, objective value, or invariant candidate. Djex reifies finite choices and preserves exact identities. Lean checks the resulting term and, when requested, the behavioral theorem.

## 3.2 Generalizing the existing behavioral envelope

Djex already has a generic `BehavioralProblem` and associated-observation layer. The best extension is not to replace it with a broad type class immediately, but to add a closed existential package around domain-specific checked values.

```
data BehavioralDomainId = BehavioralDomainId
  { domainName    :: Name
  , domainVersion :: Natural
  }

data SomeBehavioralDomain source candidate =
  forall d. SomeBehavioralDomain
```

```
{ domainId              :: BehavioralDomainId
, domainSourceAdmission :: Limits -> source
                         -> Either DomainSourceError (Source d)
, domainSealSession     :: Inventory -> Source d
                         -> Either (SessionError d) (Session d)
, domainSealCandidate   :: Session d -> VerifiedOrigin candidate
                         -> Either (CandidateError d) (Problem d)
, domainLowerQuery      :: Problem d
                         -> Either (LoweringError d) (Query d)
, domainReplayModel     :: Problem d -> ModelInput d
                         -> Either (ReplayError d) (ReplayResult d)
, domainProofObligation :: Problem d
                         -> Either (ProofError d) LeanObligation
}
```

Listing 3.1: Sketch of a first-class behavioral-domain package.

The existential type keeps each domain's source, session, problem, query, model and evidence types nominally distinct while allowing generic orchestration. A Haskell type class can still implement the dictionary internally, but a first-class record has practical advantages:

- domain selection can be decoded from a versioned contract without an open-world orphan-instance mechanism;
- the exact function bundle can enter a domain implementation fingerprint;
- experimental domains can remain package-private while sharing the orchestration layer;
- no generic code needs a type-family branch for every domain;
- tests can inject deliberately restricted domains without constructing a global instance.

The record must not expose constructors for checked values. Existing domain modules should continue to own the only sealers and evidence producers.

## 3.3 Seven semantic artifacts

Each domain should expose the following conceptual pipeline, even when several artifacts are represented by one opaque value.

1. **Source contract.** Passive, bounded syntax supplied by Leant or an embedding client. It can name source declarations but carries no authority that the names resolve.
2. **Checked contract.** The source syntax after type, role, nominal-family, provider-law, and normalization checks against an exact inventory.
3. **Checked session.** The inventory, semantic model, provider assumptions, interpretation policy, resolver authority, and domain version sealed atomically.
4. **Candidate problem.** One exact verified typed candidate associated with one checked contract and session, including all actual provider laws used by symbolic interpretation.
5. **Solver query.** A bounded typed plan and canonical bytes derived from the problem. The query is reproducible and cannot outlive its problem association.
6. **Observation.** Raw status and bounded artifact associated with the query and solver execution identity. It has no semantic authority.

7. **Evidence or proof.** A replayed counterexample, bounded validation receipt, domain-established receipt, or Lean-checked theorem. Each records the exact authority it does and does not provide.

This is deliberately compatible with the current Length flow. The generic layer should first wrap Length without changing its bytes or error precedence, then host new domains.

## 3.4 A separate selection boundary

The existing post-verification boundary proves that a behavioral assessor supplied a permutation of the exact input receipts. Hard constraints need a different nominal result:

```
data BehavioralSelectionPolicy
  = PreserveOnUnknown
  | RejectOnUnknownExplicitly
  | RequireLeanProof

data BehavioralSelectionBatch candidate = BehavioralSelectionBatch
  { selectedCandidates :: [Verified candidate]
  , rejectedCandidates :: [BehavioralRejection candidate]
  , undecidedCandidates :: [BehavioralUnknown candidate]
  }

withBehavioralSelectionInput
  :: VerificationBatch candidate
  -> (forall epoch. BehavioralSelectionInput epoch candidate -> result)
  -> result

sealBehavioralSelectionBatch
  :: SelectionLimits
  -> BehavioralSelectionPolicy
  -> BehavioralSelectionInput epoch candidate
  -> [SelectionDecision epoch candidate]
  -> Either BehavioralSelectionError (BehavioralSelectionBatch candidate)
```

Listing 3.2: An additive boundary for behaviorally selected candidates.

A decision handle must be minted from the exact verification batch, just like the current post-verification candidate handle. The seal checks one and only one decision per input occurrence, forbids foreign epochs, and validates that a rejection contains an evidence receipt whose candidate fingerprint matches that occurrence. Unknown policy is explicit. This boundary must not be represented as a permutation with missing elements, because omission would otherwise lose the proof of why a candidate disappeared.

## 3.5 Generic orchestration, domain-specific semantics

The following facilities are good candidates for shared infrastructure:

- contract acquisition limits, file admission, version tags and canonical fingerprints;
- solver process admission, executable identity, deadlines, cancellation, causal stream framing and bounded response parsing;
- typed QF_LIA, bit-vector, datatype, array, sequence, optimization and fixedpoint command syntax;
- raw observation association and artifact limits;

- counterexample-bank ownership, LRU policy, candidate replay scheduling and result statistics;
- sketch selector allocation, model decoding, exact-model blocking and assumption-literal tracking;
- cache identities, execution profiles, diagnostics and observability metrics.

The following must remain domain-specific:

- which source types and term-graph forms are modeled;
- the meaning of opaque values, functions, effects, constructors and recursion;
- provider-law vocabulary and admissibility;
- normalization, exactness and approximation direction;
- conversion from a candidate graph to symbolic semantics;
- model-input decoding and independent replay;
- what constitutes a complete bounded validator;
- the Lean proposition and proof skeleton emitted for positive authority.

**Boundary condition**

A generic SMT expression type should not become a generic semantic value. Sharing the typed syntax of Z3 theories is useful. Sharing the interpretation of a list, state transition, partial function, or recursive call without an explicit domain is not.

## 3.6 Architecture layers and dependency direction

A practical module decomposition is shown in figure 3.2.

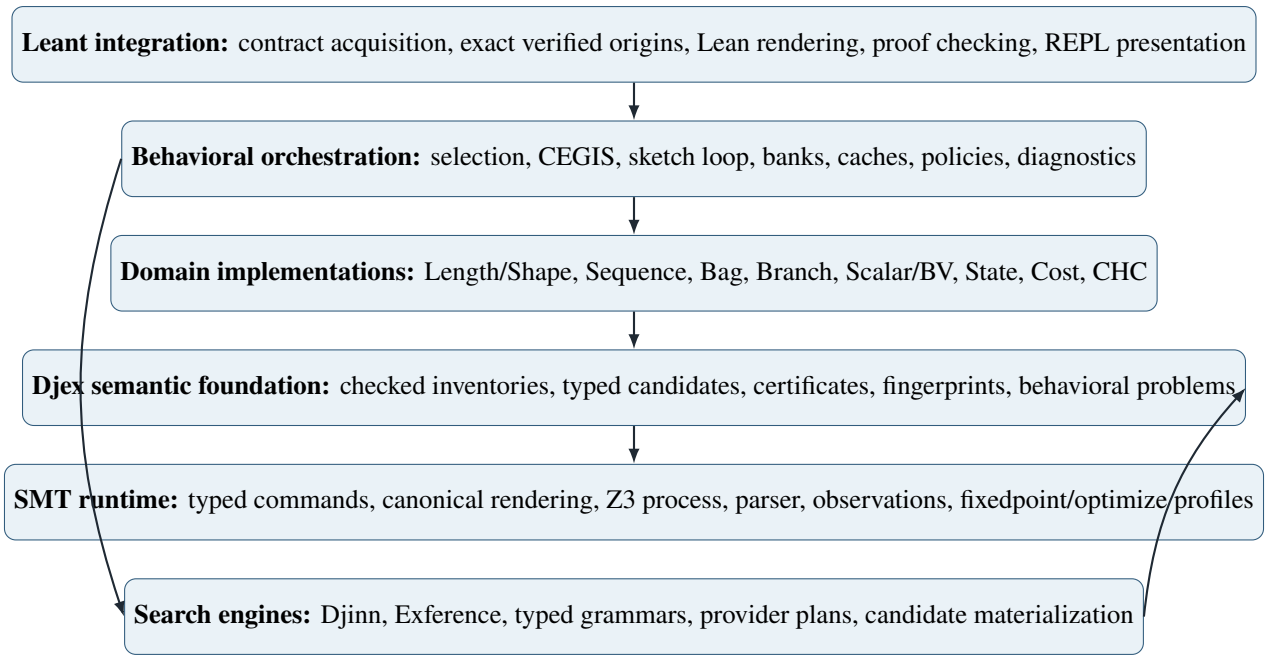


Figure 3.2: Recommended layers. Arrows indicate use of checked lower-level capabilities, not transfer of semantic authority.

The search engines should not depend directly on Z3 process code. They depend on an orchestration callback which can answer questions such as “does this sample reject the candidate?”, “give me a finite sketch model”, or “is this prefix provably completion-infeasible?” The domain and SMT layers implement those answers.

Chapter 4

# Behavioral contract language

## 4.1 Why a contract language is needed

A command-line flag such as “preserve length” is enough for one demonstration but not for a synthesis system. The contract must state at least:

- the exact target and its source-ordered argument roles;
- the semantic domain and its version;
- preconditions and postconditions;
- which observations of inputs and results are legal;
- any explicit examples or forbidden examples;
- exact source identities of modeled datatypes and providers;
- assumed provider transfer laws and their required class obligations;
- whether recursion, case analysis, partiality, overflow, or effects are modeled;
- the requested positive-proof policy and unknown policy;
- search preferences which are not hard behavioral requirements.

The language should be declarative and solver-neutral. A source file describes the intended behavior. It does not contain SMT-LIB snippets, Z3 parameters, graph identifiers, private Djex names, or proof receipts.

## 4.2 A versioned outer envelope

A proposed outer JSON envelope is shown in listing 4.1. JSON is not semantically privileged; it is merely practical for the existing file-acquisition model. The checked in-memory source should be a nominal Haskell type.

```
{
  "format": "leant-behavior-contract",
  "version": 1,
  "domain": { "name": "sequence", "version": 1 },
  "target": {
    "arguments": [
      { "name": "xs", "role": "sequence-input" }
    ],
    "result": { "role": "sequence-result" }
```

```
  },
  "requires": [ "true" ],
  "ensures": [
    { "eq": [ { "len": "result" }, { "len": "xs" } ] },
    { "forall-index": {
        "name": "i",
        "range": [0, { "len": "xs" }],
        "body": {
          "eq": [
            { "nth": ["result", "i"] },
            { "nth": ["xs", { "sub": [{"len":"xs"}, 1, "i"] }] }  // xs[len(xs) - 1 - i]
          ]
        }
    }}
  ],
  "provider-laws": [],
  "proof": { "policy": "lean-required" },
  "search": {
    "prefer": ["smaller-term", "fewer-providers"],
    "unknown": "preserve"
  }
}
```

Listing 4.1: Illustrative versioned contract envelope.

The outer version governs field grammar and default behavior. The domain version governs semantic interpretation. Execution profiles and Z3 parameters are separate activated policy files, as they are for Length today; a passive contract must not grant process-launch authority.

A remark on versioning is needed because the current Length entrances deliberately carry none. Since the pinned revision, LEANT's startup and contract-only schemas are each a single versionless document: one `format` literal, one `rankingDomain` discriminator and one nested object, with no migration decoder and no compatibility promise (section 1.3). That is the right choice for an experimental single-domain entrance: a version field whose only reader is a rejection branch is dead weight. The envelope above asks for versions for a different reason. In a contract language spanning several domains, `domain.version` is a *semantic-model identifier*: it enters session and problem fingerprints so that two contracts with identical text under different domain versions are different problems, and it lets a domain change its exactness statement without silently changing the meaning of stored samples, nogoods and proof receipts. The outer `version`, by contrast, only matters if more than one field grammar is ever accepted at once. If the project keeps its present policy, the outer version can be dropped in favor of the same format-literal-plus-discriminator shape Length now uses, and the domain version can be folded into the domain identifier; nothing else in this proposal depends on numeric file versions.

## 4.3 Contract core

The common core should be intentionally small:

$$
\begin{aligned}
C &::= \text{requires}(F) \mid \text{ensures}(F) \mid \text{example}(E) \mid \text{forbid}(E),\\
F &::= \top \mid \bot \mid F \wedge F \mid F \vee F \mid \neg F \mid F \Rightarrow F \mid R(T_1, \ldots, T_n),\\
T &::= \text{input}(i) \mid \text{result} \mid \text{literal} \mid \text{domainTerm}(\ldots).
\end{aligned}
$$

Only Boolean connective structure and target references are common. Arithmetic operations, sequence indexing, bag lookup, constructor tags, state projections, costs, and recursive relations belong to a domain or to an explicitly imported observation algebra. The parser may use one JSON AST, but checking must elaborate every domain term into a nominal typed representation.

## 4.4 Target roles and source correspondence

Current Length contracts already demonstrate why target roles must be explicit. A function argument may be:

- an observed value whose model contributes solver variables;
- an unobserved opaque value which can be forwarded but not inspected;
- a callback or provider whose applications are modeled relationally;
- a ghost input used only by a proof contract;
- a finite-control input such as an enumeration or bit-vector;
- a state or trace value modeled by a dedicated relation;
- unsupported, in which case the contract is inapplicable.

A role vector is source ordered and arity checked against the exact normalized target. It is not inferred from a type constructor name. Two arguments with the same Lean type may have different roles, and a role may carry additional source identity. For example, a sequence role records the exact modeled family and constructors, while a bit-vector role records width and signedness policy.

## 4.5 Preconditions, partial specifications, and domains of application

Behavioral synthesis frequently uses partial specifications. A sorting function may be required to behave only when a comparator is a strict order. A division helper may require a nonzero divisor. A parser may be specified only for well-formed input. A contract therefore has a semantic shape

$$\forall \vec{x}.\ P(\vec{x}) \Longrightarrow Q(\vec{x}, f(\vec{x})).$$

The counterexample query is

$$P(\vec{x}) \wedge \neg Q(\vec{x}, f(\vec{x})).$$

The contract checker must distinguish three questions:

1. Is $P$ syntactically and semantically admissible in this domain?
2. Is the application domain nonempty under the selected model?
3. Is $Q$ satisfied for every modeled input in that domain?

A vacuous contract is not the same as a satisfied nonvacuous contract. The existing Length applicable-domain receipts already retain applicable assignment counts; that discipline should become generic. A positive result should report whether nonvacuity was established, assumed, bounded, or unknown.

## 4.6 Examples as first-class constraints

Examples serve four distinct purposes and should not be conflated.

**Concrete examples.**
Exact source values and expected results, executable or reducible by Lean. These can become direct Lean tests and Z3 ground constraints.

**Symbolic examples.**
A finite set of abstract atoms, lengths, states, or function-table entries. These are CEGIS samples inside a domain, not necessarily source values.

**Negative examples.**
Inputs on which a particular result or observation is forbidden.

**Metamorphic examples.**
Relations among multiple calls, such as idempotence $f(f(x)) = f(x)$, commutation, monotonicity, or fusion.

Examples should be fingerprinted separately from universal clauses so a CEGIS session can grow the example bank without changing the source contract identity. The complete synthesis-session identity includes the ordered or canonicalized bank.

## 4.7 Provider laws

A provider law is an explicit assumption describing only the observations a domain needs. It is not a theorem about source evaluation unless separately proved. A general provider-law source might have this shape:

```
data ProviderLawSource d = ProviderLawSource
  { providerSourceName      :: SourceName
  , providerArgumentRoles   :: [ProviderArgumentRole d]
  , providerResultRole      :: ProviderResultRole d
  , providerPrecondition    :: Formula d ProviderVariable
  , providerTransfer        :: Relation d ProviderVariable
  , providerTrustClass      :: ProviderTrustClass
  }

data ProviderTrustClass
  = AssumedUniformLaw
  | AssumedConditionalOnGroundDischarge
  | LeanProvedLaw LeanTheoremName
```

Listing 4.2: General shape of an explicit provider law.

The current Length distinction between context-free assumed laws and laws conditional on exact ground class discharge should be retained. A new `LeanProvedLaw` class can point to a theorem which Leant checks in the active environment and whose elaborated type is matched against the domain-generated law proposition. The proof theorem then enters the session fingerprint by exact source identity and type, not merely by name.

## 4.8 Hard requirements versus preferences

Contracts should separate hard requirements from ranking preferences.

- **Hard requirements** enter the counterexample formula and may reject a candidate when a checked counterexample is replayed or a Lean proof obligation fails.
- **Soft behavioral preferences** enter a deterministic scoring or Optimize layer but never turn an otherwise valid candidate into a contract violation.
- **Syntactic preferences** such as term size, provider count, wildcard use, recursion depth, or source-name ratings are owned by the grammar/search layer.
- **Presentation preferences** affect which equivalent spelling is shown and must not alter candidate semantics.

A useful rule is that only the hard requirement fingerprint participates in the behavioral problem identity. Soft and syntactic preferences participate in a separate search-policy identity. This permits re-ranking an established candidate set without invalidating behavioral evidence.

## 4.9 Quantification policy

Unrestricted quantifiers are a recipe for unpredictable SMT behavior and ambiguous exactness. The contract source should expose only domain-owned bounded forms:

- finite enumeration over constructor tags or a declared finite type;
- bounded integer or natural ranges;
- a sequence index guarded by $0 \le i < \mathrm{len}(s)$;
- one symbolic element atom used for bag-count equality;
- first-order universal variables in a CHC rule;
- explicitly finite function tables for examples;
- no general quantifier over Lean types or higher-order functions.

A domain may lower a bounded universal by expansion, an index universal by a fresh counterexample index, or a recursive universal property by CHCs. The checked contract records which transformation was used and whether it is exact.

## 4.10 Exactness descriptors

Every checked contract should carry a closed exactness descriptor, for example:

```
data SemanticExactness
  = ExactForModeledSourceFragment
  | ExactWithinFiniteDomain FiniteDomainReceipt
  | OverApproximation CompletionOverApproximationId
  | UnderApproximation WitnessUnderApproximationId
  | ConditionalOnProviderLaws [ProviderLawFingerprint]
  | HeuristicObservationOnly HeuristicReason
```

Listing 4.3: Illustrative exactness vocabulary.

In practice exactness is multidimensional. A candidate may be interpreted exactly for length but only under an assumed provider law; a sequence model may be exact for bounded lists but only a finite-atom abstraction for element equality; a cost model may ignore sharing; a CHC relation may over-approximate evaluation. The internal

representation should therefore be a product of independently named claims rather than one enum once multiple domains exist.

Chapter 5

# What Z3 should be used for

## 5.1 Z3 is a reasoning engine, not the source language

Z3 can solve rich combinations of first-order theories, expose models and unsat cores, optimize objectives, and run fixedpoint engines. It does not know Lean's binder scoping, implicit arguments, universe constraints, type-class search, source nominal identities, or which generated term graph a rendered expression came from. Those facts belong to Djex and Leant.

The productive use of Z3 is therefore to solve a carefully bounded mathematical problem derived from a checked synthesis artifact. The main uses are:

1. find an input that violates a candidate's contract;
2. select among finite grammar productions in a typed sketch;
3. prove a finite-choice completion problem infeasible and learn a nogood;
4. optimize a discrete or arithmetic objective after hard constraints hold;
5. infer coefficients, thresholds, branch guards, permutations, or finite tables;
6. solve or refute CHC safety queries for recursive relational semantics;
7. minimize or explain conflicting contract clauses via assumption literals and cores;
8. propose invariants or lemmas for subsequent Lean checking.

## 5.2 Incremental scopes and assumptions

Z3's push/pop scopes and check-with-assumptions API are directly useful for CEGIS. A synthesis session has a stable base:

- declarations for grammar selectors and semantic variables;
- well-formedness constraints for the sketch;
- stable provider and datatype axioms admitted by the domain;
- accumulated sample constraints.

Each candidate check adds a temporary goal or a set of active production assumptions. The session can then pop to the stable base without reconstructing every declaration. Assumption literals are preferable when the system needs an unsat core or wants to retract a subset of choices without changing declarations.

The current live Length protocol deliberately resets and serializes query transactions under a strong causal owner. A generic incremental sketch protocol should be a new execution profile, not an informal extension of the Length check/value state machine. Its fingerprint must include scope discipline, assumption order, model requests, core settings, objective priority, and all Z3 parameters.

## 5.3 Models and counterexamples

A satisfiable counterexample query is the most dependable and useful Z3 result, provided the model is treated as untrusted input:

1. decode only the exact requested symbols under explicit byte, node, numeral and collection limits;
2. reject missing, duplicate, ill-sorted, noncanonical or out-of-range values;
3. map solver integers back to naturals, indices, tags, finite atoms, or bit-vectors through a checked domain decoder;
4. replay the exact candidate problem independently;
5. require the precondition to hold and the postcondition to fail in replay;
6. bind the receipt to the exact problem and query fingerprints;
7. only then add the sample to a counterexample bank or reject a candidate.

The existing Length stack already follows this pattern. Every future domain should reuse the generic parser and association machinery but implement its own model decoder and replay.

## 5.4 Unsatisfiable cores

Z3 can extract cores from tracked assertions or assumptions, although a returned core is not guaranteed minimal [18]. There are four valuable uses.

### 5.4.1 Contract diagnostics

Track each hard clause, provider assumption, finite-domain axiom, and target-role constraint with a stable literal. If the contract is inconsistent before any candidate is considered, a core identifies a bounded subset responsible for the contradiction. Leant can report source spans and offer a deterministic deletion-based minimization pass under a separate budget.

### 5.4.2 Grammar nogoods

When a sketch is infeasible under a set of production choices, a core over choice assumptions yields a learned clause:

$$\neg(a_{i_1} \wedge \cdots \wedge a_{i_k}).$$

This nogood can be attached to the exact grammar fingerprint and reused while the sample bank only grows. It must not be reused after changing provider inventory, semantic domain, production meaning, or candidate-size bounds.

### 5.4.3 Explanation of impossible combinations

A user may request mutually incompatible properties, such as “reverse” and “preserve every index” for lists of length greater than one. A core can identify the conflicting clauses and a domain can ask Z3 for a small witness to their incompatibility.

### 5.4.4 Proof search guidance

When a generated Lean proof obligation is discharged by a tactic such as `omega`, the corresponding solver core can help choose which hypotheses to include in the emitted theorem or which invariant clauses are essential. The core is advisory; the final minimized theorem is rechecked by Lean.

## 5.5 Optimization and MaxSMT

Z3’s optimization interface supports integer, real and bit-vector objectives and weighted soft constraints, with lexicographic objectives by default [19]. It is useful after hard behavior has been encoded. Potential objectives include:

- minimize AST node count or weighted source cost;
- minimize provider calls, case splits, duplicated inputs, or recursive calls;
- minimize the maximum observed output size or symbolic cost;
- maximize the number or weight of soft examples satisfied;
- maximize agreement with a preferred semantic profile;
- lexicographically minimize branch count, then term size, then provider rating.

There are two implementation strategies.

**Native Optimize.**
Compact and expressive, especially for soft clauses. The complete objective order and priority mode must be fingerprinted, and result reproducibility should be tested across the pinned Z3 release.

**Iterative SAT.**
Add a cardinality or arithmetic bound, solve, and tighten it. This is easier to integrate with ordinary assumption cores and often gives clearer deterministic progress. It should be the initial implementation for syntactic size; native Optimize can follow for genuinely multiobjective cases.

Optimization never establishes behavior. It chooses among models satisfying the hard encoding. Every materialized candidate still passes Lean elaboration and behavioral replay/proof.

## 5.6 Algebraic datatypes

Z3 algebraic datatypes can represent finite syntax trees, lists, trees, tuples and enumerations [20]. They are useful in two different roles.

1. **Object semantics.** A domain may represent a bounded or first-order source datatype directly and define observations such as size or constructor tags.
2. **Sketch syntax.** A finite grammar can be represented by an AST datatype and constrained by a typing relation.

The second role should be used cautiously. Djex already has a checked typed grammar and exact candidate graph. Re-encoding all typing rules as recursive Z3 predicates duplicates a trusted boundary and complicates rank-N

terms. For the first typed-sketch implementation, selector variables over a Djex-prepared DAG are preferable to a free Z3 AST. A datatype AST becomes attractive later for domains with small first-order grammars or when Synthesiz3-style symbolic term construction is evaluated.

## 5.7 Sequences and arrays

Z3 sequences provide concatenation, length, extraction, indexing and related operations over an arbitrary element sort [21]. They can make bounded sequence contracts concise, but two caveats matter:

- out-of-bounds `seq.nth` is under-specified (the guide says the result is treated as a fresh variable), so every index observation must carry and preserve an explicit bounds guard;
- solver behavior for quantified sequence properties can be less predictable than a domain-specific array-plus-length encoding.

A sequence domain should therefore support at least two lowering profiles:

**Native sequence profile.**
Use `Seq E`, `seq.len`, concatenation and guarded `seq.nth`. Suitable for concrete counterexamples and concise exact bounded models.

**Array-plus-length profile.**
Represent a sequence by $(n, a)$ where $n \in \mathbb{N}$ and $a : \mathbb{Z} \to E$, with every read guarded by $0 \le i < n$. This profile interacts naturally with Skolem counterexample indices and bag-count abstractions.

The profile is part of the encoding fingerprint. A model decoded under one profile cannot be replayed under the other without a domain-owned conversion.

## 5.8 Bit-vectors

Bit-vectors are essential for precise synthesis of low-level arithmetic [22]. They model fixed-width wraparound exactly and distinguish signed from unsigned comparisons through operators. A scalar domain should make width, signedness, shift semantics, division-by-zero policy, and source coercions explicit.

A useful first application is synthesis of branchless or saturating operations over `UInt32`/`Int32`-like Lean types or custom fixed-width structures. Z3 chooses masks, shifts, constants and conditional forms; Lean re-elaborates the term; a generated bit-vector theorem is checked with `bv_decide` where possible. This is a domain in which solver semantics and Lean proof automation can align particularly well.

## 5.9 Recursive functions versus constrained Horn clauses

Z3 supports recursive function definitions by iterative unfolding, but the official guide explicitly notes that this is not induction and only decides a limited class [23]. Recursive-function unfolding is useful for bounded counterexample generation; it is not the right positive-proof foundation for general recursive synthesized programs.

Spacer, Z3's property-directed fixedpoint engine, accepts Horn clauses over arithmetic and Boolean domains and also works with arrays, algebraic datatypes and bit-vectors [24]. It is a better fit for relational semantics of recursive programs:

$$\begin{aligned}
\mathrm{Eval}_f(x, y) &\leftarrow \mathrm{Base}(x, y),\\
\mathrm{Eval}_f(x, y) &\leftarrow \mathrm{Step}(x, x', z), \mathrm{Eval}_f(x', y'), \mathrm{Combine}(z, y', y),\\
\mathrm{Bad} &\leftarrow \mathrm{Eval}_f(x, y), P(x), \neg Q(x, y).
\end{aligned}$$

A query for `Bad` can find a counterexample or infer an inductive invariant. Even then, the fixedpoint result is a solver observation unless the invariant is translated into a Lean lemma and checked. The proposed CHC path therefore has two outputs: a replayable concrete trace for refutation, or an invariant candidate for proof generation.

## 5.10 Synthesiz3 as an adjacent research direction

The official Z3 documentation repository contains the artifact of Synthesiz3 [25], published at FMCAD 2025, which encodes synthesis specifications of the shape

$$\forall \vec{c}.\ \exists y.\ \forall \vec{u}.\ F(\vec{c}, y, \vec{u})$$

using an extended SMT-LIB command, `assert-synth`, together with a `:uncomputable` option naming the symbols $\vec{u}$ that may appear in the specification but not in the synthesized term. This work is relevant because it demonstrates synthesis-oriented reasoning on top of Z3, especially when specifications contain symbols that may be used logically but not in the synthesized term.

For LeanT/Djex it should be treated as a research backend, not the first production dependency. The typed-sketch selector encoding proposed here is smaller, uses the existing Z3 process protocol, and keeps term grammar ownership in Djex. A later experimental backend can translate a first-order checked grammar and contract into Synthesiz3's problem shape and compare quality and performance on the same benchmark corpus.

## 5.11 What not to rely on

**Boundary condition**

Do not design the architecture around general quantifier elimination, unrestricted recursive-function proofs, minimal unsat cores, stable model shapes, or an assumption that all solver versions choose the same optimum. These are useful capabilities, not semantic contracts. Every relied-upon profile needs a bounded protocol, explicit parameters, observed capability, and versioned fallback.

Chapter 6

# Integration strategies: from filtering to synthesis

## 6.1 Level 1: hard post-verification filtering

The smallest useful extension is an opt-in hard behavioral filter over already verified candidates. It changes no term grammar and can be implemented first for the existing Length domain.

### 6.1.1 Algorithm

For each callback-accepted candidate in source order:

1. Try the existing exact handoff. An inapplicable candidate receives the configured inapplicability decision: preserve, reject because the contract is mandatory, or place in an undecided tier.
2. Replay every counterexample already in the query-scoped bank. A replayed violation rejects the candidate without opening Z3.
3. Run any complete pure validator enabled by the domain. A replayed violation rejects. A complete positive receipt may accept according to policy.
4. If necessary, lower the exact candidate problem and run the solver.
5. On `sat`, decode and replay. A reproduced violation rejects the candidate and inserts the input into the bank. A failed replay is an atomic assessment failure, not a non-counterexample.
6. On `unsat`, either retain the candidate as *solver-undisproved*, request a Lean proof obligation, or accept only under an explicitly configured external-solver trust policy.
7. On `unknown`, timeout or resource exhaustion, apply the explicit unknown policy. The default is preserve.

```
filterCandidate :: Domain d
                -> SelectionPolicy
                -> CounterexampleBank d
                -> VerifiedCandidate
                -> IO (Decision d, CounterexampleBank d)
filterCandidate domain policy bank verified =
  case sealProblem domain verified of
    Left refusal -> pure (onRefusal policy refusal, bank)
    Right problem ->
      case replayBank domain problem bank of
        Violation receipt bank' -> pure (Reject receipt, bank')
        NoViolation bank' ->
          case validatePure domain problem of
```

```
            Established receipt -> pure (AcceptEstablished receipt, bank')
            Violation receipt   -> pure (Reject receipt, insert receipt bank')
            Inconclusive        -> runLive domain policy problem bank'
```

Listing 6.1: Core hard-filter loop.

### 6.1.2 Why this is more than ranking

A ranking stage can be observationally conservative because the candidate remains visible. A filter changes the result set, so three additional properties are required:

- each rejection is associated with the exact verified occurrence and carries replayed evidence;
- omission and reassociation are checked by a nominal selection seal;
- unknown and inapplicable results are policy-visible and cannot disappear as ordinary rejection.

The first user-visible syntax can be additive:

```
:synth --behavior-contract /abs/preserve-length.json -- TYPE
:synth --behavior-mode filter --behavior-contract /abs/contract.json -- TYPE
:synth --behavior-mode prove --behavior-contract /abs/contract.json -- TYPE
```

Listing 6.2: Illustrative Leant command forms.

Existing `--length-contract` continues to mean ranking unless an explicit behavior mode is selected, preserving compatibility.

## 6.2 Level 2: counterexample-guided candidate enumeration

Hard filtering becomes CEGIS when counterexamples learned from one candidate are used to constrain later candidates. The simplest loop does not ask Z3 to construct terms:

1. enumerate the next type-correct candidate from Djinn or Exference;
2. let Lean verify it;
3. replay the current bank;
4. if it survives, ask the domain for a new counterexample;
5. on a replayed counterexample, add the sample and continue enumeration;
6. on positive proof or the configured stopping criterion, return the candidate.

This can eliminate large families cheaply. For `List a -> List a`, a single sample of length one eliminates constant-empty and many partial rebuilds; a length-two atom sample separates identity, reverse, head duplication and tail selection; a state-bind sample where the intermediate state differs from the initial state eliminates the classic wrong threading candidate.

### 6.2.1 Bank identity and lifetime

A counterexample bank is not a bag of integers. It is scoped to:

$$(\text{domain session}, \text{contract}, \text{target}, \text{provider-law basis}, \text{model profile}).$$

The candidate fingerprint is intentionally absent: the bank is meant to cross candidates. Every replay still produces a fresh candidate-specific receipt. A bank entry contains only the minimum domain input necessary for replay, plus provenance and usage statistics. It does not cache a solver verdict, query, candidate, or proof.

Banks can have three lifetimes:

**Batch-local.**
Exactly the current Length style, safest and easiest.

**Command-local.**
Shared across all engine batches and progressive bounds in one `:synth` command. Recommended first CEGIS scope.

**Persistent.**
Saved in a synthesis companion keyed by the exact target/contract/session fingerprint. Useful later, but must survive environment changes only through exact identity checks.

### 6.2.2 Replay ordering

A good deterministic order is:

1. samples which most recently rejected a candidate;
2. samples with the highest historical rejection count;
3. smaller samples under the domain's canonical complexity order;
4. remaining samples in insertion order.

This generalizes the existing small MRU bank while retaining predictable behavior. The bank has independent limits on count, encoded bytes, per-sample width, and cumulative replay work.

### 6.2.3 Semantic signatures

For each candidate, evaluate the vector of observations on the current bank:

$$\sigma_B(c) = \big(\mathrm{obs}(c, b_1), \ldots, \mathrm{obs}(c, b_k)\big).$$

This signature is search metadata, not identity. It supports:

- suppressing candidates behaviorally identical on all known samples;
- retaining only the cheapest representative of a sample-equivalence class;
- selecting a candidate likely to split the largest remaining class;
- measuring information gain from a new counterexample;
- interleaving structural novelty with behavioral novelty.

A structurally distinct candidate must not lose its typed graph or authority merely because a signature table keeps another representative. The table stores candidate handles, never replaces candidate identity.

## 6.3 Level 3: Z3-completed typed sketches

Typed sketches are the first mode in which Z3 directly participates in term construction, following the broad sketching and syntax-guided-synthesis tradition while retaining Djex ownership of the grammar [26, 27]. The crucial choice is where the grammar comes from.

**Recommendation**

Djex should prepare a finite, type-correct selector DAG; Z3 should select alternatives and solve semantic parameters. Do not initially encode the full type system or binder calculus as Z3 datatypes.

### 6.3.1 Sketch representation

A sketch consists of typed nodes in topological order. Each node has one result type and a finite set of productions already checked by Djex. A production may reference earlier nodes, local binders, globals, constructors, primitive constants, or child holes. For example:

```
data SketchNode local ty = SketchNode
  { sketchNodeId       :: SketchNodeId
  , sketchNodeType     :: ty
  , sketchAlternatives :: NonEmpty (SketchAlternative local ty)
  }

data SketchAlternative local ty
  = UseLocal local
  | UseGlobal CheckedGlobalOccurrence
  | Apply SketchNodeId SketchNodeId
  | Lambda Binder (SketchNodeId)
  | Construct ConstructorId [SketchNodeId]
  | Case SketchNodeId [SketchBranch local]
  | Literal LiteralFamily ParameterSlot
  | Primitive PrimitiveId [SketchNodeId]
```

Listing 6.3: Conceptual typed selector-DAG representation.

The actual representation should reuse or derive from the checked `TermGraph` vocabulary, with occurrence and certificate identities preserved when a selected global is materialized. A sketch alternative is admitted only if every reference, type application, constructor family and provider occurrence has a checked source origin.

### 6.3.2 Selector encoding

For node $i$ with alternatives $a_{i,1}, \ldots, a_{i,m_i}$ introduce selector $s_i$ with

$$0 \le s_i < m_i.$$

Alternatively use one-hot Booleans if unsat cores over individual alternatives are more important. The semantics of node $i$ on sample $b$ is expressed as

$$v_{i,b} = \mathrm{ite}(s_i = 0, E_{i,0,b}, \mathrm{ite}(s_i = 1, E_{i,1,b}, \ldots)).$$

A shared DAG avoids exponential AST duplication. It also permits Z3 to choose common subexpressions explicitly. Acyclicity is guaranteed by Djex's topological references rather than solved by Z3.

### 6.3.3 Literal and guard synthesis

Many useful terms require constants or branch thresholds. Parameter slots can have domain-specific finite or arithmetic ranges:

- integer coefficients in $[-K, K]$;
- bit-vector masks of fixed width;

- small natural literals;
- constructor tags or tuple projections;
- a permutation of argument indices;
- affine branch guards $a_1x_1 + \cdots + a_nx_n \leq k$;
- bounded finite lookup tables.

These parameters are model values, not source terms, until checked decoding and materialization. The decoder must prove range and representation constraints before creating a literal or primitive application in the candidate graph.

### 6.3.4 Sample constraints

For every bank sample, assert the hard contract over the root node's symbolic value. Z3 finds selectors and parameters satisfying all current samples. Djex decodes the model into one exact graph. Lean re-elaborates the rendered candidate. The domain then asks for a counterexample to the universal contract.

```
sketchCegis :: SketchSession d -> IO (SketchOutcome d)
sketchCegis session = loop initialBank initialSolver
 where
  loop bank solver = do
    modelResult <- solveSketch solver bank
    case modelResult of
      SketchUnsat core -> pure (NoSketchWithinBounds core)
      SketchUnknown why -> pure (SketchInconclusive why)
      SketchModel raw -> case decodeMaterialize raw of
        Left badModel -> pure (SketchProtocolFailure badModel)
        Right candidate -> do
          verified <- verifyInLean candidate
          case verified of
            Rejected diagnostic ->
              loop bank (blockExactModel raw solver)
            Accepted receipt -> do
              check <- findCounterexample receipt
              case check of
                ReplayedCounterexample cex ->
                  loop (insert cex bank) (assertSample cex solver)
                LeanProofAccepted proof ->
                  pure (FoundCandidate receipt proof)
                NoCounterexampleObserved status ->
                  pure (CandidateSurvives receipt status)
```

Listing 6.4: Typed-sketch CEGIS loop.

### 6.3.5 Blocking

When a model yields a candidate that Lean rejects, block the exact selector/parameter assignment. When it yields a behaviorally failing candidate, the new counterexample usually excludes it automatically, but an exact block is still useful if the sample abstraction does not distinguish the model. Stronger blocks include:

- selector-level exact model blocking;
- structural alpha-equivalence blocking;

- semantic-signature blocking over the current bank;
- core-derived nogoods for an infeasible partial assignment;
- symmetry-breaking constraints for commutative operators or interchangeable holes.

Only exact-model blocking is required for correctness. Stronger blocks are optimizations and must not remove a candidate outside the proved equivalence or infeasibility class.

### 6.3.6 Grammar generation strategies

Three grammar sources should be supported progressively.

**Bounded normal forms.**
Generate beta-normal, eta-long structural alternatives for the target: introduce the target's binders, then at each hole offer the locals of the right type, one-layer case splits on locals of a modeled family, constructor applications and tuple formation, up to a node bound. No provider is needed. Best for plumbing terms and logic such as state threading or `Option.bind`.

**Exference frontier extraction.**
Reuse Exference's rated providers and partial search states to create a goal-relevant grammar: the prepared provider occurrences Exference would try at a goal, already type-checked and certificate-associated, become the alternatives of a node of that goal type, and its rating order becomes the alternative order. Best for library composition, where the useful terms are applications of a few loaded functions.

**Template families.**
Domain-specific skeletons such as folds, maps, affine arithmetic, bit tricks, case splits, or state-threading forms, with parameter slots where the domain expects constants, coefficients or selectors. Best for deep behavior with a small combinatorial shell, such as saturating arithmetic or a clamp.

A sketch identifies the grammar source and all bounds in its fingerprint. "No sketch" means no satisfying term in that finite grammar, not uninhabitation of the type or impossibility of the contract in Lean.

## 6.4 Level 4: solver-guided partial-search pruning

Waiting for a complete candidate can waste most of the search budget. Partial pruning asks whether any completion of a search state could satisfy the known samples or universal abstraction.

### 6.4.1 Completion summaries

For each typed hole $h$, a domain computes a relation $S_h$ over the observations any permitted completion may produce. The summary is useful when it is an over-approximation:

$$\{\mathrm{obs}(t) \mid t \text{ completes } h\} \subseteq S_h.$$

Substituting $S_h$ into the partial term yields an over-approximation $S_p$ of all complete candidates extending prefix $p$. If

$$S_p \wedge \text{contract samples}$$

is unsatisfiable, then no completion satisfies the samples and the prefix may be pruned.

Examples of sound summaries include:

- interval or affine bounds on list length;

- possible constructor-tag sets;
- finite sets of argument dependencies;
- bit masks of bits known zero, known one, or unknown;
- possible state-source identities (initial, intermediate, arbitrary);
- lower and upper bounds on cost;
- a regular tree grammar of possible shapes.

### 6.4.2 A worked example

Take the target `List a -> List a`, the contract $\text{len}(f(xs)) = \text{len}(xs)$, and a bank holding one sample, the empty list ($n = 0$). Suppose Exference is at the partial state

$$\texttt{fun xs => List.cons ?h}_1\ \texttt{?h}_2, \qquad \texttt{?h}_1 : a, \quad \texttt{?h}_2 : \texttt{List a}.$$

The Size/Shape domain knows nothing about what will fill $\texttt{?h}_2$ except its type, so its summary is the weakest sound statement: the completion has *some* length $t \in \mathbb{N}$. Composing that with the exact step case gives the prefix summary $\text{len}(\text{result}) = 1 + t$, and the pruning query on the bank is

$$1 + t = 0 \ \wedge \ t \geq 0,$$

which is unsatisfiable. Every completion of the prefix returns a nonempty list on the empty input, so the prefix and its whole subtree can be pruned, and the receipt records the summary ("$t \geq 0$ for hole $\texttt{?h}_2$"), the exact bank identity, and the grammar fingerprint that scoped it.

The polarity rule matters here. Had the domain instead summarized $\texttt{?h}_2$ by the completions it happened to enumerate first, say "one of the locals", giving $\text{len}(\text{result}) = 1 + n$, the query on a bank holding a length-one sample would read $1 + 1 = 1$ and would also be unsatisfiable, but that would prune a prefix which the completion $\texttt{?h}_2 := \texttt{[]}$ satisfies on that sample. An under-approximation can report "here is a completion that works", never "no completion works". The over-approximate summary $t \geq 0$ correctly leaves the length-one sample satisfiable ($t = 0$).

### 6.4.3 Nominal approximation polarity

The API should make an accidental direction reversal difficult:

```
data OverApproximation d prefix
-- constructor private to the domain

data UnderApproximation d prefix
-- constructor private to the domain

pruneFromUnsat
  :: OverApproximation d prefix
  -> UnsatObservation
  -> Either ReplayMismatch (PruningReceipt d prefix)

witnessFromSat
  :: UnderApproximation d prefix
  -> SatObservation
  -> Either ReplayMismatch (CompletionWitness d prefix)
```

Listing 6.5: Nominal distinction between completion approximations.

A generic `Approximation` flag is too easy to misuse. Distinct nominal types and distinct query constructors should be used.

### 6.4.4 Assumption-literal learning

Associate each search decision with an assumption literal. When an over-approximate completion query is unsatisfiable, extract a core and learn a nogood over only the decisions that matter. Exference can use the nogood as a memoized forbidden partial pattern. The receipt is scoped to the exact grammar, sample bank and summary-domain fingerprints.

### 6.4.5 Fairness

Pruning must not accidentally convert a heuristic search into a false completeness claim. Search status should record:

- whether every pruned prefix carried a checked pruning receipt;
- whether some prefixes were dropped by ordinary queue/resource limits;
- whether the domain refused summaries for some constructs;
- whether the sample bank, rather than the universal contract, justified pruning;
- whether the remaining grammar was explored exhaustively.

Only a fully explored finite grammar with checked pruning receipts can produce "no satisfying sketch within this grammar." It still cannot produce a general nonexistence theorem unless the grammar is known complete for the type and contract fragment.

## 6.5 Level 5: recursive synthesis with CHCs

Recursive synthesis requires separating three tasks:

1. choose a recursive program schema;
2. prove or refute its behavioral relation;
3. prove termination in Lean.

Z3/Spacer can assist with the second task. Djex and Lean must own the first and third.

### 6.5.1 Relational encoding

For a candidate schema, generate a relation `Eval` rather than a recursive SMT function. Each constructor case becomes a Horn rule. Provider calls become relations with explicit assumed or proved laws. The bad state is a rule whose body contains the evaluation relation, precondition and negated postcondition.

For list reverse with an accumulator, for example:

$$\mathrm{RevAcc}(\mathrm{nil}, a, a).$$
$$\mathrm{RevAcc}(\mathrm{cons}(x, xs), a, r) \leftarrow \mathrm{RevAcc}(xs, \mathrm{cons}(x, a), r).$$

A sequence or bag abstraction can replace full element lists when that is the selected domain. Spacer may prove safety by finding an invariant such as

$$\text{len}(r) = \text{len}(xs) + \text{len}(a)$$

or a corresponding bag equation.

### 6.5.2 Invariant extraction and Lean proof

A solver-produced invariant should be normalized into the domain's proposition language, checked to mention only admitted symbols, and translated to a Lean lemma. Leant then attempts a proof skeleton:

```
theorem revAcc_length_invariant
    (xs acc : List a) :
    (revAcc xs acc).length = xs.length + acc.length := by
  induction xs generalizing acc with
  | nil => simp [revAcc]
  | cons x xs ih =>
      simp [revAcc, ih, Nat.add_assoc, Nat.add_comm, Nat.add_left_comm]
```

Listing 6.6: Illustrative generated invariant proof skeleton.

The exact proof will vary, but the architecture is stable: Spacer proposes a predicate; the domain translates it; Lean proves it; the final candidate theorem uses it. If proof generation fails, the invariant remains advisory and may still guide search.

### 6.5.3 Termination

CHC safety says nothing by itself about Lean's acceptance of a recursive definition. Candidate materialization must use a structurally recursive schema, a well-founded measure, or an existing recursive provider. Lean's termination checker and any generated `termination_by`/`decreasing_by` proof remain authoritative.

## 6.6 Level 6: specification and law inference

A later, clearly labeled experimental mode can use Z3 to infer unknown contract parameters or provider summaries:

- infer affine length transfer coefficients from examples;
- infer which input state a candidate uses;
- infer a branch threshold or bit mask;
- infer a finite permutation or projection;
- infer candidate invariants for recursive code;
- infer a weakest condition within a restricted predicate grammar.

These outputs are *proposals*. A provider law becomes usable only after explicit user acceptance or a Lean theorem establishing it. The system must never convert "fits the current examples" into an assumed universal law automatically.

## 6.7 Hybrid scheduling

The modes above should not form one mandatory pipeline. A scheduler can interleave them:

1. run cheap sample replay on ordinary Djinn/Exference candidates;
2. periodically ask a typed sketch solver for a model satisfying all samples;
3. when the sketch solver is unsatisfiable, widen grammar bounds or resume ordinary enumeration;
4. use semantic signatures to avoid rechecking redundant candidates;
5. launch expensive universal or CHC checks only for candidates that survive cheap stages;
6. request Lean proof generation only for the best surviving candidates.

Fairness requires explicit lanes and budgets. A fast sketch lane must not starve complete Djinn search in a fragment where a definitive logical result is possible. Conversely, a provider-heavy Exference lane should not consume the whole behavioral budget before the compact sketch lane runs.

Chapter 7

# Behavioral domains

## 7.1 Domain 1: generalized size and shape

The existing Length domain should become the seed of a broader Size/Shape family, not be discarded. Its current implementation already provides exact finite-spine identities, role-aware symbolic inputs, modeled zero/step cases, provider transfers, scalar and pair results, bounded evaluation, QF-LIA lowering, and extensive finite applicable-domain validation.

### 7.1.1 Generalization axes

A versioned Size/Shape domain can add the following observations while retaining the same trust pattern:

- size of arbitrary structurally checked unary recursive families;
- constructor counts for multi-constructor datatypes;
- tree height, minimum depth and leaf count;
- product-field sizes of arbitrary fixed arity;
- sums and maxima of component sizes;
- whether a result is empty/nonempty or belongs to a bounded shape class;
- affine and piecewise-affine provider transfers;
- exact one-layer case behavior for each modeled constructor.

For a datatype

$$D\ \alpha = C_1(\vec{p}_1, \vec{r}_1) \mid \cdots \mid C_k(\vec{p}_k, \vec{r}_k),$$

a size model declares one equation per constructor, for example

$$\mathrm{size}(C_j(\vec{p}, r_1, \ldots, r_m)) = c_j + \sum_i w_{j,i}\,\mathrm{size}(r_i).$$

The source family and recursive fields must come from an exact checked schema. The caller may choose coefficients only within a domain version that defines their meaning.

### 7.1.2 Shape abstractions

Full algebraic values are often unnecessary. A finite shape abstraction can use:

- root constructor tag;

- bounded depth-$d$ constructor tree;
- size interval;
- constructor-count vector;
- nullary/non-nullary and singleton/multiple classifications.

These abstractions are useful for early pruning. They are exact only within the declared observation. They do not imply element preservation or source evaluation behavior.

### 7.1.3 Recommended first extension

Add an exact *constructor-tag plus size* domain for `Option`, `Except`, user enums and list-like families. This unlocks precise `Option.bind`, error propagation, nonempty preservation, and branch behavior with minimal new theory beyond integers and finite tags.

## 7.2 Domain 2: sequences and indexed behavior

A sequence domain observes order, not merely size. Its core checked value is an abstract finite sequence over an element sort $E$. For polymorphic Lean elements, $E$ is an uninterpreted sort whose equality means only equality in the model.

### 7.2.1 Core observations

$$\begin{aligned} \mathrm{len}(s) &\in \mathbb{N}, \\ \mathrm{nth}(s,i) &\in E \quad \text{under } 0 \le i < \mathrm{len}(s), \\ \mathrm{concat}(s,t) &\in \mathrm{Seq}(E), \\ \mathrm{slice}(s,o,n) &\in \mathrm{Seq}(E), \\ \mathrm{reverse}(s) &\in \mathrm{Seq}(E). \end{aligned}$$

A first implementation does not need a primitive reverse. It can express the law with a fresh counterexample index:

$$0 \le i < n \wedge \mathrm{nth}(r,i) \neq \mathrm{nth}(x, n-1-i).$$

### 7.2.2 Candidate interpretation

The symbolic interpreter can model:

- spine zero and step constructors as empty and singleton-prepend;
- exact zero/step case analysis;
- tuples and lets;
- providers with explicit sequence transfers such as identity, append, reverse, map and filter abstractions;
- opaque elements which may be forwarded and compared for modeled equality but not called or inspected;
- index-safe primitives admitted by a domain-specific provider law.

Map requires a relational element provider. Rather than interpret an arbitrary Lean function, assign an uninterpreted unary symbol $F : E \to E'$ to the exact callback argument and state the result law pointwise. The model assumes totality because SMT uninterpreted functions are total; the contract descriptor must record that this is a mathematical extensional model, not a claim about source strictness, effects or nontermination.

### 7.2.3 Bounded versus unbounded modes

**Finite-spine bounded mode.**
Enumerate lengths up to $N$ and finite atoms up to $A$. Exact for that finite abstraction; suitable for fast counterexamples and exhaustive bounded receipts.

**Index-counterexample mode.**
Use unbounded length and one or more symbolic indices. Exact for quantifier-free index laws after guarded Skolemization when candidate semantics is expressible in the chosen theory.

**CHC mode.**
Use relational list semantics and inductive invariants for recursive providers or candidates. Appropriate for unbounded proof attempts.

## 7.3 Domain 3: bags, multisets, and permutations

Length preservation cannot distinguish `id`, `reverse`, rotations, duplicating one element while deleting another, or many other transformations. Bag equality adds element multiplicity while ignoring order.

### 7.3.1 Finite-atom encoding

For a bounded atom set $A = \{a_1, \ldots, a_k\}$, represent a bag by counts

$$\mathrm{count}_s(a_j) \in \mathbb{N}.$$

Bag equality is a conjunction over all atoms. This is exact for the finite-atom abstraction and works well in CEGIS: Z3 can invent an atom assignment that exposes duplication or loss.

### 7.3.2 Symbolic-witness encoding

For generic elements, introduce one symbolic witness $e : E$ and ask for

$$\mathrm{count}_r(e) \neq \mathrm{count}_x(e).$$

This transforms a universal bag-equality law into an existential counterexample witness. Candidate interpretation must provide a count transfer for constructors and providers. The encoding is exact for the modeled count semantics if recursive count equations are handled exactly or relationally.

### 7.3.3 Array encoding

A bag can be represented as an SMT array $E \to \mathbb{Z}$ constrained nonnegative at observed indices. Constructor insertion becomes a store/update. This is concise for finite candidate expressions, but universal nonnegativity and extensional equality require care. Prefer pointwise witness queries and domain replay rather than asking for full array models.

### 7.3.4 Combined specifications

Useful contracts layer observations:

- **Permutation:** length equality plus bag equality.
- **Stable filter:** result is an order-preserving subsequence plus a predicate law.
- **Partition:** bag union equals input, output bags are disjoint by predicate, and each output preserves relative order.

- **Deduplication:** result is a subsequence, contains each observed atom at most once, and preserves membership.

Each layer should produce a separate subreceipt so diagnostics can say whether a candidate preserved size but violated multiplicity or order.

## 7.4 Domain 4: constructor and branch behavior

Many important functions are characterized by their constructor cases. Examples include `Option.bind`, `Except.map`, defaulting, error propagation, and one-layer destructors.

A branch domain observes:

- input constructor tags;
- output constructor tag;
- selected payload provenance;
- which callback/provider occurrence was invoked;
- whether a callback is unused, called once, or may be called multiple times in the modeled graph;
- branch-specific value relations.

### 7.4.1 Finite tag encoding

Constructor tags form a finite datatype or integer enumeration. A candidate graph with exact case authority can be interpreted as an if-then-else over tags. Payloads are opaque tokens with stable provenance classes. This allows a contract such as:

$$\begin{aligned} \mathrm{bind}(\mathrm{none}, f) &= \mathrm{none}, \\ \mathrm{bind}(\mathrm{some}(x), f) &= f(x). \end{aligned}$$

The second equation is relational: the result must equal the modeled application of the exact callback token to the exact payload token. No source evaluation is performed.

### 7.4.2 Call traces

A finite call-trace abstraction can record a bounded sequence of provider-call events:

$$\mathrm{Call}(p, \vec{v}), \quad \mathrm{Return}(p, r).$$

It can distinguish short-circuiting, callback duplication, ignored arguments and state-threading. The trace is a mathematical graph-evaluation trace, not a claim about Lean's runtime evaluation order unless a domain version defines one for a restricted pure fragment.

## 7.5 Domain 5: scalar arithmetic

Scalar arithmetic is a natural domain for precise synthesis because Z3 can solve equations over integer, rational, nonlinear and bit-vector terms, while Lean has strong proof tactics for many resulting obligations.

### 7.5.1 Natural and integer profiles

A QF-LIA profile supports:

- affine expressions and inequalities;
- min, max, monus, positive-literal quotient and modulo through checked lowering;
- piecewise-affine branches;
- bounded coefficient synthesis;
- monotonicity or range counterexamples with two calls;
- lexicographic cost objectives.

The current Length implementation already contains a sophisticated natural arithmetic normalizer and applicable-domain machinery. That code should be factored into a reusable arithmetic observation library only where its semantics is genuinely domain-independent.

### 7.5.2 Nonlinear arithmetic

Nonlinear integer arithmetic can be useful for polynomial templates, but solver completeness and performance are less predictable. Treat it as a separate profile with strict time and degree bounds. A successful model can synthesize coefficients; an unsat result remains heuristic unless a Lean tactic proves the resulting polynomial theorem.

### 7.5.3 Relational properties

Properties such as monotonicity, injectivity on a bounded range, Lipschitz bounds or idempotence require multiple symbolic calls:

$$\begin{aligned} x_1 \le x_2 &\Longrightarrow f(x_1) \le f(x_2), \\ f(x_1) = f(x_2) &\Longrightarrow x_1 = x_2, \\ f(f(x)) &= f(x). \end{aligned}$$

A candidate interpreter must either duplicate a pure symbolic evaluation with shared provider symbols or refuse providers/effects that make such extensional duplication unsound under the model.

## 7.6 Domain 6: machine arithmetic and bit-vectors

A bit-vector domain models exact fixed-width values. It is suitable for:

- constant and mask synthesis;
- branchless min/max/absolute-value idioms;
- saturating add/subtract;
- population count or parity templates;
- byte packing and unpacking;
- sign extension, truncation and rotate/shift operations;

- low-level range checks and overflow detection.

The grammar should consist of a curated safe primitive set with exact Lean renderers. Z3 chooses operator selectors, constants and wiring. The target proof proposition is translated to Lean bit-vector or fixed-width arithmetic and checked by `bv_decide`, normalization, or a library theorem. Undefined or source-dependent operations are excluded unless their Lean semantics is modeled exactly.

## 7.7 Domain 7: state transitions and relational effects

The motivating state-bind type

$$(S \to A \times S) \to (A \to S \to B \times S) \to S \to B \times S$$

has multiple inhabitants. A correct bind must satisfy

$$\begin{aligned} (a, s_1) &= f(s_0), \\ (b, s_2) &= g(a, s_1), \\ \mathrm{bind}(f, g, s_0) &= (b, s_2). \end{aligned}$$

This is expressible without knowing the internal meanings of $S$, $A$ or $B$. Model them as uninterpreted sorts and $f$, $g$ as uninterpreted total functions into tuple datatypes. Congruence is enough to distinguish passing $s_1$ from passing $s_0$.

### 7.7.1 Provenance relational semantics

A lightweight alternative to full uninterpreted functions tracks symbolic provenance terms:

$$\begin{aligned} F_s &= \pi_2(F(s_0)), \\ F_a &= \pi_1(F(s_0)), \\ G_b &= \pi_1(G(F_a, F_s)), \\ G_s &= \pi_2(G(F_a, F_s)). \end{aligned}$$

The candidate result is compared structurally to $(G_b, G_s)$. A wrong candidate using $G(F_a, s_0)$ differs in the free term algebra and is refuted by congruence unless the model happens to equate the states; Z3 can choose a separating interpretation.

### 7.7.2 General state contracts

The same domain can describe:

- reader environment threading;
- writer log concatenation;
- lens get-put and put-get laws;
- transaction state invariants;
- parser state/position progression;
- finite protocol transitions;

- relational refinement between two state implementations.

Effects are modeled only as explicit mathematical state. IO, exceptions, nondeterminism or nontermination require separate domains or explicit monadic providers with laws.

## 7.8 Domain 8: cost and resource behavior

A cost domain attaches a symbolic nonnegative cost to term-graph operations and provider calls. It can express:

- number of provider calls or constructor allocations;
- maximum recursion depth under a recurrence;
- affine time or memory estimates;
- upper bounds conditional on input sizes;
- multiobjective preferences after semantic correctness.

Cost semantics is especially sensitive to sharing and evaluation strategy. The first version should define cost on the *term graph interpretation*, not claim actual Lean runtime cost. Lets may share a node; duplicated applications may count twice; opaque providers use explicit assumed cost transfers. The contract displays this model prominently.

A proof-grade cost theorem is possible only when the source program is instrumented or related to a formal cost semantics in Lean. Until then, cost remains a search objective or modeled contract.

## 7.9 Domain 9: recursive relational behavior

This domain packages the CHC path from section 6.5. It should be introduced only after nonrecursive exact domains are stable. Its checked contract identifies:

- the exact recursive candidate schema and calls;
- a relational abstraction of values;
- base and step rules;
- admitted provider relations;
- candidate termination authority in Lean;
- a bad-state predicate;
- an invariant grammar acceptable for Lean translation.

The main outputs are concrete counterexample traces or candidate invariants. A fixedpoint “safe” status by itself remains an observation. A Lean proof of the invariant and theorem upgrades the result.

## 7.10 Domain 10: higher-order finite observations

Arbitrary higher-order extensional equality is out of scope, but useful finite observations are possible. A function argument can be modeled by:

- an uninterpreted function over first-order modeled sorts;
- a finite table on the current sample atoms;
- a symbolic callback with exact call/return events;

- an algebraic law supplied as an explicit provider contract;
- a universally quantified first-order relation in CHC mode.

The chosen observation class must be explicit. For example, treating a callback as an SMT uninterpreted function assumes extensionality, totality and purity in the mathematical model. This is excellent for distinguishing argument wiring; it is not a theorem about evaluation effects.

**Recommendation**

Prioritize domains in this order: generalized size/tag, state/provenance, sequence/index, scalar/bit-vector, bag/permutation, then recursive CHC. This order maximizes reuse of the existing graph interpreter and QF-LIA/Z3 stack while delivering behavior that types alone fail to express.

Chapter 8

# Worked synthesis scenarios

## 8.1 From `List a -> List a` to exact intent

The type

$$\forall \alpha.\ \mathrm{List}\,\alpha \to \mathrm{List}\,\alpha$$

admits many programs. Behavioral constraints can be layered to move from a broad family to a precise implementation.

### 8.1.1 Layer 1: length preservation

$$\forall xs.\ \mathrm{len}(f(xs)) = \mathrm{len}(xs).$$

This rejects every candidate whose output length can differ from the input length: constant empty or singleton results, functions that drop or duplicate elements without compensation, and truncations. It retains identity, reverse, rotations, arbitrary permutations, element replacement, and duplicate/delete combinations whose net length change is zero.

The existing Length domain can already rank candidates by counterexamples to this law. The first proposed hard-filter milestone can reject replayed violators.

### 8.1.2 Layer 2: bag preservation

$$\forall xs, e.\ \mathrm{count}(f(xs), e) = \mathrm{count}(xs, e).$$

Together with length, this characterizes permutation behavior in the mathematical list model. It rejects element replacement and duplicate/delete compensation. It retains identity, reverse and permutations.

### 8.1.3 Layer 3a: identity

$$\forall xs, i.\ 0 \le i < \mathrm{len}(xs) \implies f(xs)[i] = xs[i].$$

This pins the result pointwise. In a parametric pure total setting it is equivalent to list equality, but the domain should prove or assume only the selected sequence semantics.

### 8.1.4 Layer 3b: reverse

$$\forall xs, i.\ 0 \le i < n \Longrightarrow f(xs)[i] = xs[n-1-i], \qquad n = \mathrm{len}(xs).$$

A sketch grammar can include:

- `xs`;
- `List.reverse xs`;
- `List.map g xs`;
- `List.foldr step init xs`;
- one-layer cases and constructors;
- selected loaded providers with explicit sequence laws.

On two finite-atom samples, Z3 may immediately choose `List.reverse`. The provider occurrence is materialized with its exact scheme and Lean source name. Lean elaborates the term. A proof-mode request then emits the theorem

```
def it1 {a : Type u} : List a -> List a := List.reverse

theorem it1_contract {a : Type u} (xs : List a) :
    it1 xs = xs.reverse := by
  rfl
```

Listing 8.1: Desired proof artifact for a reverse candidate.

If the candidate is a hand-built fold, the theorem requires list induction; the proof generator can use the provider laws and standard simp lemmas.

## 8.2 Correct state threading

Consider the type:

```
(S -> Prod A S) -> (A -> S -> Prod B S) -> S -> Prod B S
```

A type-directed synthesizer may produce both:

```
-- correct
fun f g s0 =>
  match f s0 with
  | (a, s1) => g a s1

-- wrong: reuses the initial state
fun f g s0 =>
  match f s0 with
  | (a, _) => g a s0
```

Listing 8.2: Correct and incorrect inhabitants of the same state-bind type.

### 8.2.1 SMT model

Declare uninterpreted sorts $S, A, B$ and functions

$$F : S \to A \times S, \qquad G : A \times S \to B \times S.$$

For symbolic $s_0$, define

$$a = \pi_1(F(s_0)), \quad s_1 = \pi_2(F(s_0)), \quad expected = G(a, s_1).$$

The wrong candidate denotes $G(a, s_0)$. Ask whether they differ. Z3 can choose a model with $s_0 \neq s_1$ and distinguish the two applications by congruence.

### 8.2.2 Replay

The replay need not produce actual Lean values of arbitrary $S, A, B$. It re-evaluates the exact symbolic term graph in the same free algebra and checks that the candidate result term differs from the contract term under the decoded finite model. The receipt says "relational state-threading counterexample in the uninterpreted total-function model," not "concrete runtime counterexample."

### 8.2.3 Lean proof

A proof-grade contract can quantify over the actual functions and state and define the expected expression in Lean:

```
theorem bind_contract
    (bind : (S -> Prod A S) -> (A -> S -> Prod B S) -> S -> Prod B S)
    (hbind : bind = fun f g s0 =>
      match f s0 with | (a, s1) => g a s1) :
    forall f g s0,
      bind f g s0 =
        (match f s0 with | (a, s1) => g a s1) := by
  intro f g s0
  simp [hbind]
```

Listing 8.3: Lean theorem shape for state bind.

For a synthesized closed term, Leant substitutes the candidate directly and asks `rfl`, `simp`, or structural tactics to prove the equation.

## 8.3 Option bind

The type

$$\forall \alpha\, \beta.\ \text{Option}\, \alpha \to (\alpha \to \text{Option}\, \beta) \to \text{Option}\, \beta$$

admits the always-none function. A branch contract specifies:

$$f(\text{none}, k) = \text{none},$$
$$f(\text{some}(x), k) = k(x).$$

The branch domain uses exact constructor tags and payload provenance. A two-sample sketch problem suffices: one `none` sample and one `some` sample with a callback table that returns a distinguishable token. The correct case expression survives; the constant `none` candidate fails the second sample.

The generated proof is direct case analysis:

```
theorem optionBind_contract {a b : Type}
    (x : Option a) (k : a -> Option b) :
    it1 x k = Option.bind x k := by
  cases x <;> rfl
```

This is an ideal early vertical slice because it exercises exact constructor cases, callback provenance, sketch selectors, Lean materialization and a simple proof artifact without recursive semantics.

## 8.4 Partition into two lists

Suppose the target is

$$(\alpha \to \mathbb{B}) \to \mathrm{List}\,\alpha \to \mathrm{List}\,\alpha \times \mathrm{List}\,\alpha.$$

A useful behavioral contract states:

1. every input occurrence appears in exactly one output bag;
2. every element in the first output satisfies the predicate;
3. every element in the second does not;
4. relative order within each output is preserved;
5. output lengths sum to input length.

The current Length pair domain already models two list-spine fields and can express the length sum. A staged extension can add:

- a predicate callback represented as an uninterpreted Boolean function;
- bag counts parameterized by a symbolic element;
- a stable-subsequence sequence relation;
- a recursive fold/filter provider law or CHC relation.

This example is valuable as an integration benchmark because each domain catches a different defect: dropping elements, duplicating elements, swapping sides, ignoring the predicate, and reversing one side.

## 8.5 Map with a behavioral callback law

For

$$(\alpha \to \beta) \to \mathrm{List}\,\alpha \to \mathrm{List}\,\beta,$$

the contract is

$$\begin{aligned} \mathrm{len}(r) &= \mathrm{len}(xs), \\ r[i] &= g(xs[i]) \quad (0 \le i < \mathrm{len}(xs)). \end{aligned}$$

The callback $g$ becomes an uninterpreted function $G : E_\alpha \to E_\beta$. A candidate which ignores $g$ can build a `List b` only from constructors and providers that need no $\beta$ value, such as the constant empty list, and the length clause already rejects those; a candidate which applies $g$ to the wrong element, applies it twice to one element, or reverses the list is distinguishable by a symbolic index and atom model.

When the selected candidate is `List.map`, Leant can generate a proof by reflexivity after unfolding the alias. A fold-based implementation may require induction and standard list lemmas. Provider laws can be either assumed or linked to Lean theorems.

## 8.6 Saturating addition over fixed-width words

A bit-vector sketch can synthesize a saturating unsigned addition:

$$\mathrm{satAdd}(x, y) = \begin{cases} 2^w - 1, & x + y \text{ overflows}, \\ x + y, & \text{otherwise}. \end{cases}$$

A grammar may include addition, unsigned comparison, conditional, all-ones, zero, and selected masks. Z3 solves selector and constant variables exactly in QF_BV. The model is decoded into a typed primitive graph, rendered to Lean, and checked. The universal theorem is then discharged by `bv_decide` or a bit-vector normalization tactic.

This scenario demonstrates the full positive path with no provider-law assumptions and a decidable finite domain.

## 8.7 Tree transformation with a size invariant

For a recursive tree function, a size contract may be simple even when value semantics is not:

$$\mathrm{size}(f(t)) = \mathrm{size}(t).$$

A finite-depth shape model can quickly reject candidates that drop or duplicate subtrees. A CHC relation can then reason about unbounded size. Spacer may propose an affine invariant. Leant translates it into an induction lemma over the exact user-declared family and asks Lean to prove it. The candidate is accepted as proof-grade only after that theorem succeeds and the recursive definition passes termination checking.

Chapter 9

# Concrete API and codebase changes

## 9.1 Preserve the current ownership seams

The proposed implementation should preserve the following existing seams exactly where possible:

- `Verified candidate` remains the callback-acceptance receipt. It is not upgraded in place into behavioral evidence.
- `ExactTypedVariantOrigin` remains the Leant-owned route from accepted text to exact typed candidate and semantic provenance.
- `TypedCandidate` remains the Djex-owned carrier for compatibility output, checked graph availability, and certificate association.
- `BehavioralProblem` remains the generic association of domain, inventory, encoding, candidate and problem fingerprints.
- raw `SolverObservation` and `BehavioralObservation` remain honest report vocabularies with no automatic evidence conversion.
- the current Length session/contract/problem/query/evaluator/live stack remains the reference implementation and compatibility adapter.
- process launch authority remains separate from passive contracts.

Generalization should occur by adding orchestration and domain packages around these values, not by making their constructors public or weakening their nominal roles.

## 9.2 Proposed Djex modules

table 9.1 lists an additive module plan. Names are illustrative but follow the current repository's organization.

Table 9.1: Proposed Djex module additions.

| Module | Responsibility |
|---|---|
| `Language.Haskell.Synthesis.Semantic.Domain` | First-class closed domain dictionary, domain identifiers, exactness claims, common refusal classes and existential packaging. |

| Module | Responsibility |
|---|---|
| `Language.Haskell.Synthesis.Semantic.Contract` | Solver-neutral outer contract envelope, target-role correspondence, hard/soft separation, clause/source-span identities and canonical contract fingerprints. |
| `Language.Haskell.Synthesis.Semantic.Selection` | Nominal post-verification selection input, exact decision seal, rejection/unknown/established batches and evidence association checks. |
| `Language.Haskell.Synthesis.Semantic.Counterexample` | Domain-neutral bank owner, bounded insertion/LRU policy, sample provenance, replay scheduling, bank fingerprints and statistics. |
| `Language.Haskell.Synthesis.Semantic.CEGIS` | Candidate-level CEGIS state machine, progressive bounds, stopping criteria and lane scheduling. |
| `Language.Haskell.Synthesis.Semantic.Sketch` | Checked typed selector DAG, grammar identities, selector/literal slots, materialization, exact model blocking and sketch result envelope. |
| `Language.Haskell.Synthesis.Semantic.Sketch.SMTLib` | Generic finite selector and sample-constraint lowering over domain-provided symbolic node semantics. |
| `Language.Haskell.Synthesis.Semantic.Approximation` | Nominal over-/under-approximation wrappers, prefix-query association, pruning receipts and core-derived nogoods. |
| `Language.Haskell.Synthesis.Semantic.ProofObligation` | Backend-neutral proof proposition, named hypotheses, provider-law assumptions and exact candidate/contract association. |
| `Language.Haskell.Synthesis.Semantic.SizeShape` | Generalized Length-compatible size/tag domain; initially an adapter over current Length scalar and pair implementations. |
| `Language.Haskell.Synthesis.Semantic.Sequence` | Sequence roles, expressions, formulas, interpreter, finite/index profiles, model decoder and replay. |
| `Language.Haskell.Synthesis.Semantic.Bag` | Count/bag expressions, finite-atom and symbolic-witness profiles, replay and combined permutation receipts. |
| `Language.Haskell.Synthesis.Semantic.Branch` | Constructor-tag, payload-provenance and bounded callback-trace semantics. |
| `Language.Haskell.Synthesis.Semantic.Scalar` | General scalar expression/formula foundation and proof-obligation lowering; reuse of exact arithmetic normalizers where legitimate. |
| `Language.Haskell.Synthesis.Semantic.BitVector` | Width-indexed operations, fixed-width model decoding, primitive grammar and Lean proposition projection. |
| `Language.Haskell.Synthesis.Semantic.State` | Uninterpreted relational state/callback semantics and provenance replay. |
| `Language.Haskell.Synthesis.Semantic.Cost` | Explicit graph-cost model, provider cost transfers and optimization objectives. |
| `Language.Haskell.Synthesis.Semantic.CHC` | Checked relational rules, bad-state query, invariant vocabulary and exact association with a recursive candidate schema. |
| `Language.Haskell.Synthesis.Internal.SMTLib.Optimize` | Typed optimization commands/objectives and bounded response handling, independent of ordinary solver authority. |

| Module | Responsibility |
|---|---|
| `Language.Haskell.Synthesis.Internal.SMTLib.Fixedpoint` | Typed relation/rule/query commands, fixedpoint response framing and invariant/certificate artifact bounds. |
| `Language.Haskell.Synthesis.Internal.SMTLib.Theory.*` | Reusable typed syntax for arrays, datatypes, sequences and bit-vectors, without domain interpretation. |

The curated `Language.Haskell.Djex` facade should initially expose only orchestration types needed by Leant and the mature Size/Shape adapter. Experimental domains can remain focused module imports until their contracts stabilize.

## 9.3 Proposed Leant modules

Table 9.2: Proposed Leant module additions.

| Module | Responsibility |
|---|---|
| `Leant.Synth.Behavior.Contract` | Lean-facing passive contract source, exact source names, target role sources and domain selection. |
| `Leant.Synth.Behavior.Contract.File` | Versioned JSON grammar and bounded decoder; adapters for existing Length contract files. |
| `Leant.Synth.Behavior.Configuration` | Activated execution/search/proof policy, solver pinning and resource limits. |
| `Leant.Synth.Behavior.Handoff` | Generic verified-origin recovery and domain dispatch; preserves Length's exact refusal order through its adapter. |
| `Leant.Synth.Behavior.Selection` | Integration of the Djex selection seal with one verification batch. |
| `Leant.Synth.Behavior.CEGIS` | Command-local sample bank, engine/sketch scheduling and progressive bound control. |
| `Leant.Synth.Behavior.Sketch` | Translation of Lean source inventories and semantic families into Djex sketch grammars; exact rendering/materialization variants. |
| `Leant.Synth.Behavior.Proof` | Lean proposition rendering, theorem submission, tactic portfolio, proof receipt and failure diagnostics. |
| `Leant.Synth.Behavior.Presentation` | Candidate status, counterexamples, assumptions, proof strength, cores and statistics. |
| `Leant.Synth.Behavior.Session` | Snapshot companion data for persistent contracts, samples and proof artifacts under exact fingerprints. |

The existing `Leant.Synth.Length.*` hierarchy should not be renamed immediately. `Behavior.Handoff` can dispatch the Size/Shape-v1 selection to the current Length handoff. Once the generic orchestration proves stable, internal factoring can remove duplication without changing the public file schemas.

## 9.4 Core types

### 9.4.1 Behavioral source and activation

```haskell
data LeanBehaviorContractSource = LeanBehaviorContractSource
  { behaviorContractFormatVersion :: Natural
  , behaviorDomainSource          :: LeanBehaviorDomainSource
  , behaviorTargetSource          :: LeanBehaviorTargetSource
  , behaviorHardClauses           :: [LeanBehaviorClauseSource]
  , behaviorSoftPreferences       :: [LeanBehaviorPreferenceSource]
  , behaviorProviderLawSources    :: [LeanBehaviorProviderLawSource]
  , behaviorProofRequest          :: LeanBehaviorProofRequest
  }

data BehaviorExecutionPolicy = BehaviorExecutionPolicy
  { behaviorUnknownPolicy       :: UnknownPolicy
  , behaviorInapplicablePolicy  :: InapplicablePolicy
  , behaviorSolverProfile       :: SolverExecutionProfile
  , behaviorCegisLimits         :: CegisLimits
  , behaviorSketchLimits        :: Maybe SketchLimits
  , behaviorProofPolicy         :: ProofPolicy
  }
```

Listing 9.1: Proposed passive contract and activated policy split.

The source is passive and can be parsed without opening a process. The activated policy carries execution authority and is created only through the same explicit admission/pinning pattern as current Length ranking.

### 9.4.2 Candidate decisions

```haskell
data BehavioralDecision d candidate
  = CandidateRejected
      (Verified candidate)
      (CounterexampleEvidence d)
  | CandidateEstablished
      (Verified candidate)
      (EstablishedEvidence d)
  | CandidateLeanProved
      (Verified candidate)
      LeanProofReceipt
  | CandidateSurvives
      (Verified candidate)
      (PositiveObservation d)
  | CandidateUndecided
      (Verified candidate)
      BehavioralUnknown
  | CandidateInapplicable
      (Verified candidate)
      DomainRefusal
```

Listing 9.2: Candidate decision vocabulary.

Constructors should probably remain private, with domain and Lean proof modules producing nominal receipts. The type shown here illustrates the distinctions the public projections must preserve.

### 9.4.3 Counterexample bank

```haskell
data CounterexampleBank d = CounterexampleBank
```

```
  { counterexampleBankSession     :: SessionFingerprint d
  , counterexampleBankContract    :: ContractFingerprint d
  , counterexampleBankModel       :: ModelProfileFingerprint d
  , counterexampleBankEntries     :: Seq (BankEntry d)
  , counterexampleBankStatistics  :: BankStatistics
  }

data BankEntry d = BankEntry
  { bankInput         :: CounterexampleInput d
  , bankOrigin        :: CounterexampleOrigin
  , bankComplexity    :: CounterexampleComplexity d
  , bankUseCount      :: Natural
  , bankRejectCount   :: Natural
  }
```

Listing 9.3: Counterexample bank with no cached verdict authority.

No `CandidateFingerprint`, `SolverStatus`, query bytes, receipt or proof is stored in an entry. Replaying against a candidate mints fresh association.

### 9.4.4 Sketch authority

```
data CheckedSketchGrammar d local ty
-- opaque; owns exact typed alternatives and source origins

data AssociatedSketchModel d grammar
-- raw model associated with grammar, sample bank and execution profile

data MaterializedSketchCandidate d local ty
-- exact graph plus model/grammar receipt, not yet Lean-verified

materializeSketchModel
  :: CheckedSketchGrammar d local ty
  -> AssociatedSketchModel d grammar
  -> Either SketchModelError (MaterializedSketchCandidate d local ty)
```

Listing 9.4: Separation of grammar, solver model and materialized candidate.

A model cannot be decoded against a different grammar even if selector names happen to match. Grammar node and alternative coordinates are not durable identities; canonical grammar fingerprints abstract away allocation where possible while preserving semantics and order where it matters.

## 9.5 Refactoring the Length adapter

The first generic-domain implementation should be an adapter, not a rewrite:

```
lengthBehaviorDomain :: SomeBehavioralDomain
lengthBehaviorDomain = SomeBehavioralDomain
  { domainId = BehavioralDomainId "finite-spine-length" 1
  , domainSourceAdmission = decodeExistingLeanLengthSelection
  , domainSealSession = sealViaCurrentLengthSession
  , domainSealCandidate = prepareViaCurrentLengthHandoff
  , domainLowerQuery = sealViaCurrentLengthAdapter
  , domainReplayModel = replayViaCurrentLengthProblem
```

```
  , domainProofObligation = renderLengthLeanObligation
  }
```

Listing 9.5: Conceptual adapter over the current Length implementation.

The adapter should preserve existing scalar/product nominal distinctions, the exact configuration schemas, fingerprint bytes, executable policy, refusal constructors and ranking behavior. New selection/CEGIS modes consume the adapter through new code paths; ordinary ranking remains unchanged.

## 9.6 Term-graph requirements for sketches and domains

The current semantic foundation already identifies a future limitation: Djinn candidates do not yet retain the same typed graph authority as Exference because historical LJT sidecars erased source names, provider rewrites, assumption identity and nominal type structure. This has direct consequences:

- exact candidate-level behavior can initially remain Exference-only where a domain requires a graph;
- Djinn candidates can still participate in Lean proof-mode behavior by rendering the accepted term and generating a theorem, because Lean is the proof authority;
- a future Djinn graph lowerer must be built from checked proof construction before final cleanup/rendering, not reconstructed from text;
- sketch generation should reuse the shared checked grammar and Exference-origin path first;
- combined-engine exact-text deduplication must retain variant-scoped Exference authority exactly as current Length handoff does.

**Recommendation**

Do not block the behavioral roadmap on a complete Djinn typed-graph migration. Implement Exference-backed exact semantics and Lean-proof fallback first, while designing the shared graph lowerer as a parallel correctness project.

## 9.7 Provider certificates in sketch materialization

A sketch alternative that uses a provider must preserve the same information as a directly generated Exference occurrence:

- exact source owner scheme;
- source-ordered explicit type selections;
- activated constraints;
- occurrence-specific certificate row and slot handles;
- final visible-application node;
- exact ground-discharge receipts where required;
- renderer metadata for Lean named/positional type arguments.

The safest design is for the sketch grammar to contain *prepared provider occurrences*, not provider names plus free type variables. Materialization then selects an already checked occurrence plan and builds the same certificate-associated graph form Exference would have emitted. Z3 never chooses a source type by inventing

a string; it chooses among bounded checked selections or among a parameter domain whose decoded type is rechecked before graph construction.

## 9.8 SMT protocol families

The current Length live protocol is intentionally narrow. New uses should have distinct nominal protocol identities.

**Counterexample protocol.**
Reset, assert exact bad-state formula, check, request only required input values, replay. This is the current pattern.

**Incremental sketch protocol.**
Declare stable selectors, assert grammar and accumulated samples, use assumptions or push/pop, request selector/parameter values, optionally request a core.

**Optimization protocol.**
Declare hard sketch constraints and ordered objectives, check, request model and objective values. Separate response types from ordinary SAT.

**Fixedpoint protocol.**
Declare relations/variables/rules, query bad state, retrieve bounded answer or invariant text under a domain-specific decoder.

They may share the same domain-neutral process owner, lexical parser and causal driver, but should not share one phase-state type with optional commands. A state machine that can ask for an unsat core after a model-producing query or issue Optimize commands to a fixedpoint context is too permissive.

## 9.9 Solver-neutral query IR

A layered query IR avoids hand-built SMT strings:

1. theory terms: Bool, Int/Real, bit-vector, array, datatype and sequence;
2. typed commands: declarations, assertions, assumptions, objectives, fixedpoint rules;
3. domain plan: exact symbol roles and formula components;
4. canonical renderer and structural fingerprint projection;
5. bounded expected-response schema.

The renderer and fingerprint projection must consume the same typed plan. A query fingerprint should not be computed from rendered bytes alone; structural fields catch accidental renderer equivalences and keep names/roles explicit. Conversely, the exact rendered bytes should enter the execution observation so replay can detect protocol drift.

## 9.10 Caching

There are several distinct caches with different safety rules.

Table 9.3: Cache classes and required identities.

| Cache | Key | Safe payload |
|---|---|---|
| Preparation cache | Lean environment generation, target, provider-discovery policy | Prepared semantic origin/inventory projections under existing generation rules. |
| Contract cache | Inventory, domain source, limits, domain version | Checked passive contract/session only; no process handle. |
| Candidate-problem cache | Session, contract, exact typed candidate, interpretation policy | Opaque checked problem; invalidated by any graph/provider/certificate change. |
| Counterexample bank | Session, contract, target, model profile | Domain input vectors and provenance only. |
| Query cache | Problem, lowering profile and limits | Canonical query plan/bytes; no solver status. |
| Observation cache | Query, solver executable/version/profile, exact parameters | Raw associated observation; must still replay before evidence use. |
| Evidence cache | Exact problem plus domain verifier version and limits | Opaque replay/validation receipt, if its replay authority and limits are immutable. |
| Lean proof cache | Exact theorem text/environment fingerprint/backend version | Accepted elaboration receipt or saved theorem declaration; normal session replay rules apply. |
| Nogood cache | Grammar, bank, domain summary, assumptions | Checked grammar-choice nogoods only. |

Raw solver `sat` should not be cached as a counterexample. Cache the bounded observation if useful, then replay it for the exact problem. A persistent counterexample bank stores decoded inputs only after successful replay at origin, but each later candidate still replays them freshly.

Chapter 10

# Lean proof-producing integration

## 10.1 Why proof production changes the value proposition

Counterexample guidance can produce excellent programs, but the user eventually wants to know whether the behavior was established. The strongest integration is to bind each synthesized definition to a generated theorem:

```
def it1 : TARGET := CANDIDATE

theorem it1_behavior : CONTRACT_PROP it1 := by
  -- generated and Lean-checked proof
```

Listing 10.1: Candidate plus behavioral theorem as the final artifact.

This design lets Z3 remain an untrusted accelerator. A solver model finds bugs. An invariant or core guides proof construction. The Lean backend checks the final proposition in the exact environment, with actual types, universes, instances and definitions.

## 10.2 Proof-obligation generation

A domain-generated obligation contains:

- exact target type and candidate binding;
- source-order binders for contract inputs;
- precondition and postcondition rendered in Lean;
- explicit provider-law hypotheses or references to checked theorems;
- model exactness hypotheses if the domain is conditional;
- any auxiliary invariant statements;
- suggested imports and tactics;
- exact association to the candidate and contract fingerprints.

The proposition renderer is domain-specific. It must not simply pretty-print the SMT formula: natural monus, sequence indexing, bit-vector operations, constructor tests and provider relations must map to actual Lean definitions whose semantics matches the domain.

## 10.3 Proof tactic portfolio

A bounded proof portfolio can be selected by the obligation profile.

Table 10.1: Suggested Lean proof strategies by domain.

| Obligation profile | Initial tactics | Notes |
|---|---|---|
| Definitional equality | `rfl`, `simp` | Best for selected providers, projections and direct case expressions. |
| Propositional finite cases | `decide`, cases, simp | Constructor/tag contracts and finite enums. |
| Presburger arithmetic | `omega`, simp (`norm_num` where Mathlib is imported) | Natural/integer affine obligations, including monus, `min`/`max` and quotient/modulo by literals, which `omega` handles natively; `if-then-else` must be split first. Unfold only named candidate/provider definitions. |
| Bit-vector equality | `bv_decide`, simp | Requires exact width/source semantics; `bv_decide` lives in core Lean but needs `import Std.Tactic.BVDecide` in the active environment. |
| List/tree structural property | induction, simp_all, omega | Z3/Spacer may propose the invariant or generalized induction hypothesis. |
| Equality using library provider | simp [candidate, provider theorem] | Prefer checked source theorem references over re-proving provider internals. |
| Finite bounded domain | `native_decide` or explicit enumeration | Appropriate only when the theorem itself states the bound/finite type. |
| Relational state wiring | rfl, simp, cases on tuple result | Uninterpreted-function counterexamples guide choice; proof is usually definitional. |
| CHC invariant | induction/well-founded induction + arithmetic tactics | Solver invariant is translated and checked before use. |

All tactics named in the table except `norm_num` are part of core Lean at the referenced toolchain; `grind`, also in core, is a reasonable later addition to the arithmetic and structural rows once its behavior on generated obligations has been measured. The proof runner should retain Leant's present crash/restart and exact environment replay behavior. Proof-state suggestions are not authority; only successful declaration submission produces a receipt.

## 10.4 Reconstructing QF-LIA unsatisfiability in Lean

For an exact scalar candidate problem, the Z3 counterexample formula has the form

$$\exists \vec{x} \in \mathbb{N}.\ P(\vec{x}) \wedge \neg Q(\vec{x}).$$

An unsat result corresponds to

$$\forall \vec{x} \in \mathbb{N}.\ P(\vec{x}) \rightarrow Q(\vec{x}).$$

Rather than import a Z3 proof, generate this universal proposition using the same normalized expression tree and ask Lean's `omega` to prove it. This yields a practical proof-producing bridge for the current Length arithmetic fragment. The key requirements are:

- one shared normalized expression algebra with separately verified Lean and SMT renderers;

- differential tests comparing bounded evaluation of both projections;
- a pre-processing step for the one construct `omega` does not handle: it decides linear arithmetic over `Nat`/`Int` including natural subtraction, `min`/`max` and division or modulo by numeric literals, but it does not case-split `if-then-else`, so the caseLen conditional must be split first (`split <;> omega`) or rendered as two equations;
- provider transfer laws as theorem hypotheses or checked theorem references;
- exact candidate interpretation theorem connecting the synthesized term to the arithmetic result expression.

The last item is the difficult one. The current symbolic interpreter derives an arithmetic expression from a typed graph, but a Lean theorem must connect that graph interpretation to the actual term. For simple constructor/provider fragments, Leant can generate a structural proof by unfolding the candidate and provider-law theorems. This should be the first proof-producing subset.

A complete small instance shows the two halves. For the candidate

```
def it1 {a : Type u} : List a -> List a :=
  fun xs => match xs with
    | [] => []
    | x :: rest => x :: rest
```

the Length interpreter derives $r(n) = \text{caseLen}(n, 0, (n \dot{-} 1) + 1)$, and Z3 reports `unsat` for $r(n) \neq n$. The arithmetic half of the reconstructed theorem is the universal form of that query, and the connecting half relates the term to its interpretation:

```
theorem it1_len_arith (n : Nat) :
    (if n = 0 then 0 else (n - 1) + 1) = n := by
  split <;> omega

theorem it1_len {a : Type u} (xs : List a) :
    (it1 xs).length =
      (if xs.length = 0 then 0 else (xs.length - 1) + 1) := by
  cases xs <;> simp [it1]

theorem it1_contract {a : Type u} (xs : List a) :
    (it1 xs).length = xs.length := by
  rw [it1_len]; exact it1_len_arith _
```

Listing 10.2: Reconstructed obligation for a length-preserving one-layer case.

The first theorem is what Z3's `unsat` corresponds to; the second is what the symbolic interpreter silently assumed; only the third is the user-facing claim, and it is accepted by Lean, not by the solver.

## 10.5 Provider assumptions as hypotheses

Suppose the candidate uses `List.reverse` under an assumed length law. A proof-grade theorem should either:

1. reference a checked theorem such as `List.length_reverse`; or
2. expose the provider law as a theorem hypothesis in a generic obligation.

It must not present a conclusion as unconditional merely because the SMT session assumed the transfer. A useful presentation is:

```
theorem it1_contract
    (h_reverse : forall (xs : List a),
      (List.reverse xs).length = xs.length)
    (xs : List a) :
    (it1 xs).length = xs.length := by
  simp [it1, h_reverse]
```

Listing 10.3: Making a provider law explicit when no source theorem is linked.

In ordinary Lean environments the system should prefer linking the actual library theorem. Explicit hypotheses remain useful for opaque project providers.

## 10.6 Reusing Leant's existing proof machinery

LEANT already contains most of the runtime this chapter needs, and the proposal should be read as a client of it rather than a second implementation. Its `:prove` mode drives the backend's proof-state protocol: every line is a tactic run against an immutable proof state, goals reprint after each step, and a Lean-verified suggestion is computed by actually running candidate tactics against that state. The probes are ordered from quick finishers (`rfl`, `assumption`, `contradiction`, `trivial`, `decide`, `omega`) through goal-shaped structural steps (`intro`, `cases`/`obtain`, `left`/`right`, `induction`) to chained forms such as `induction l <;> simp_all [myLen] <;> omega`, which is how a one-line suggestion can already be a finished induction. `:auto` tries an ordered finisher list (`rfl`, `trivial`, `decide`, `simp`, `omega`, `exact?`, and `aesop` where the project imports it), and `:qed NAME` turns the accumulated script into a theorem saved in the session. Proof-state identifiers belong to one backend process; if that process stops, LEANT leaves prove mode and prints the script instead of submitting a stale identifier after restart.

Four consequences follow for behavioral proof mode:

1. the portfolio in table 10.1 is a domain-indexed refinement of `:auto`'s list and should run through the same proof-state protocol, inheriting its per-tactic timeouts and its rule that only an accepted tactic counts;
2. an obligation the portfolio does not close is not a dead end: `:behavior-proof` can open the generated statement in `:prove`, so the user finishes it interactively with the same suggestions, and `:qed` produces the theorem;
3. `:qed`'s save path is the natural producer of the `LeanProofReceipt` of section 10.7, because it already threads the environment generation and snapshot identity that the receipt must record;
4. the one-process rule for proof-state identifiers is exactly the crash/restart discipline a proof receipt must inherit; a receipt minted under a backend that has since restarted is replayed or invalidated by the normal environment logic, never trusted by name.

Suggestions remain advisory, as they are today. Only successful declaration submission produces a receipt.

## 10.7 Proof receipts and theorem identity

A `LeanProofReceipt` should retain:

- backend environment generation or snapshot identity;
- exact theorem source submitted;
- exact candidate definition name and body identity;
- elaborated theorem declaration name and type;

- active imports and options relevant to elaboration;
- candidate/contract/problem fingerprints;
- provider theorem identities;
- whether the theorem was saved into the session, a transient example, or a snapshot companion.

A theorem name alone is not enough because it may be shadowed or redefined after reset. The normal Leant environment-threading and snapshot logic should own durability.

## 10.8 Proof-mode user experience

A successful command might present:

```
it1  fun xs => xs.reverse
     behavior: proved in Lean
     theorem: it1_behavior
     contract: sequence/reverse-v1
     assumptions: none
     counterexamples examined: 3
     sketch models blocked: 1
```

Listing 10.4: Illustrative proof-grade synthesis output.

A non-proof positive result is labeled honestly:

```
it1  fun xs => xs.reverse
     behavior: no counterexample found
     solver: unsat for exact QF_LIA length projection
     Lean proof: not requested
     additional sequence clauses: validated on 12 symbolic samples only
```

This granularity is preferable to a single green check mark.

## 10.9 Proof failure is useful feedback

A candidate may survive SMT semantics but fail the generated Lean theorem because:

- the domain abstraction was conditional or incomplete;
- a provider law had no linked theorem;
- source reduction does not match the abstract evaluation strategy;
- the invariant is too weak for induction;
- required imports or simp lemmas are missing;
- the contract proposition was rendered incorrectly;
- the candidate is extensionally correct but the proof portfolio is insufficient.

The system should distinguish theorem *refutation* from proof-search failure. Lean elaboration of a counterexample theorem may expose a real mismatch; a tactic timeout merely leaves the claim unproved. Failed invariants can be fed back to Z3 by requesting a stronger invariant grammar or a concrete CHC trace.

Chapter 11

# Leant command surface and presentation

## 11.1 Command design

The command surface should make mode and authority explicit. An illustrative design is:

```
:synth [SYNTH-OPTIONS]
       [--behavior-contract ABSOLUTE-PATH]
       [--behavior-mode rank|filter|cegis|sketch|prove]
       [--behavior-config ABSOLUTE-PATH]
       -- TYPE
```

Mode meanings:

**rank.**
: Preserve all verified candidates and apply semantic ordering. This is the compatibility mode for current Length behavior.

**filter.**
: Reject only candidates carrying a checked rejection receipt; preserve or separate unknowns according to policy.

**cegis.**
: Interleave ordinary candidate enumeration with a command-local counterexample bank.

**sketch.**
: Enable the typed-sketch lane in addition to ordinary engines; output remains Lean-verified.

**prove.**
: Require a Lean-accepted behavioral theorem before reporting a candidate as successful; may still display unproved survivors in a separate section if configured.

The mode must be specified explicitly for any behavior that can remove candidates or launch a sketch/CHC session. A contract file alone should not silently change existing `:synth` semantics.

## 11.2 Interactive contract forms

File contracts are appropriate for rich specifications, but common cases deserve compact syntax:

```
:synth --where "len(result) = len(xs)" -- List a -> List a
:synth --law state-bind -- TYPE
:synth --law option-bind -- TYPE
```

```
:synth --example "[1,2] => [2,1]" -- List Nat -> List Nat
```

These are frontends that construct the same passive source AST as a file. They must not implement independent semantics. Names such as `state-bind` resolve to versioned built-in contract templates, and the expanded contract fingerprint is shown in diagnostics.

## 11.3 Contract inspection

Add commands analogous to type and environment inspection:

```
:behavior-check CONTRACT -- TYPE
:behavior-show CONTRACT
:behavior-explain it1
:behavior-counterexamples
:behavior-proof it1 [THEOREM_NAME]
```

`:behavior-check` performs acquisition, target correspondence and domain sealing without running synthesis or Z3. It reports exact modeled arguments, source families, provider laws, theory profile, proof strategy and all refusals. This prevents expensive debugging through a full synthesis command. `:behavior-proof` shows the generated obligation and its receipt for a named candidate; when the obligation is unproved it opens the statement in `:prove` (section 10.6) so the user can finish it interactively.

## 11.4 Candidate presentation

Each candidate should display a compact primary status and expose detail on request. A stable set of labels might be:

- proved in Lean;
- established by finite exhaustive validation;
- replayed counterexample;
- no counterexample found (solver unsat);
- validated within bounds;
- unknown: timeout/resource;
- behavior inapplicable;
- conditional on provider laws;
- model-relative relational result.

The detailed explanation includes:

- domain/model/version;
- contract clauses and source spans;
- provider assumptions actually used, not merely configured;
- exact counterexample input and simplified form;
- bounded-validation range and applicable count;

- solver profile and query ordinal;
- proof theorem or proof failure;
- sketch grammar bounds, models blocked and learned nogoods;
- whether the candidate came from Djinn, Exference, a sketch, or multiple equivalent routes.

## 11.5 Counterexample rendering

Counterexamples should be rendered at the highest trustworthy level available.

**Concrete Lean value.**
When the domain can construct a source value and Lean evaluation confirms the failure, show a runnable expression.

**Finite abstract value.**
Show list length, finite atoms, constructor tags, state symbols or bit patterns with a clear model label.

**Relational model.**
Show equations such as $F(s_0) = (a, s_1)$, $s_0 \neq s_1$, and candidate call $G(a, s_0)$ versus expected $G(a, s_1)$.

**CHC trace.**
Show the bounded rule sequence, abstract states and violated clause.

Never print opaque internal solver symbols as if they were source identifiers. The domain owns stable user-facing names.

## 11.6 Explaining rejection

A rejection explanation should answer four questions:

1. Which exact hard clause failed?
2. On what modeled input did it fail?
3. What observation did the candidate produce and what was required?
4. Which provider assumptions, if any, were involved?

For example:

```
rejected it2: state/threading-v1
  input model:
    f(s0) = (a0, s1)
    s0 != s1
    g(a0, s0) != g(a0, s1)
  candidate calls g(a0, s0)
  contract requires g(a0, s1)
  evidence: model decoded and replayed against exact typed candidate
  assumptions: f and g are total pure uninterpreted functions in this model
```

## 11.7 Explaining no result

A behaviorally constrained search can fail in several ways. The output should separate them:

- type is provably uninhabited in a complete translated fragment;
- type has candidates, but every verified candidate found was behaviorally refuted;
- finite sketch grammar is behaviorally unsatisfiable;
- search or solver bounds were exhausted;
- contract was inapplicable to all candidates;
- candidates survived but no Lean proof was found;
- contract itself is inconsistent or vacuous;
- provider laws required by every candidate were missing or unproved.

The distinction between “no satisfying term exists” and “no term found within these grammar/search/proof bounds” remains essential.

Chapter 12

# Performance, resource control, and reproducibility

## 12.1 Cost model of the pipeline

Behavioral synthesis adds several multiplicative dimensions:

$$\text{candidates} \times \text{samples} \times \text{symbolic cost} + \text{solver queries} + \text{proof attempts}.$$

Without staged checks, a rich universal query for every candidate will be too expensive. The scheduler should order work from cheapest to most discriminating:

1. exact candidate-origin and domain applicability checks;
2. replay of a small high-value sample prefix;
3. structural/semantic signature lookup;
4. pure bounded evaluators;
5. simplified solver query or origin probe;
6. full live counterexample query;
7. sketch solving or Optimize;
8. CHC query;
9. Lean proof generation.

## 12.2 Independent limits

The configuration should expose independent limits rather than one global “steps” number. At minimum:

Table 12.1: Required independent resource limits.

| Limit family | Examples |
|---|---|
| Contract acquisition | file bytes, JSON depth/nodes, clause count, identifier bytes, literal bits. |
| Target preparation | source type nodes, providers, families, constructors, exact assignment vectors, renderer variants. |
| Candidate graph | nodes, edges, occurrences, cases, pattern nodes, certificate rows, retained fingerprint bytes. |

| Limit family | Examples |
|---|---|
| Symbolic interpretation | evaluation steps, branch count, provider calls, expression/formula nodes, normalized bytes. |
| Counterexample bank | entry count, total bytes, per-entry width, replay attempts, cumulative replay steps. |
| Query construction | symbols, declarations, assertions, theory nodes, rendered bytes, fingerprint bytes. |
| Live process | launch timeout, readiness timeout, query deadline, cumulative stdout, response nodes, cleanup deadline, total usable work. |
| Sketch grammar | nodes, alternatives per node, total alternatives, holes, parameter slots, grammar bytes, sample count. |
| CEGIS | iterations, models, blocked models, learned clauses, progressive-bound rounds, total solver calls. |
| Optimization | objectives, soft clauses, objective bits, tightening rounds. |
| CHC | relations, rule count, variables/rule, invariant artifact bytes, trace depth, fixedpoint deadline. |
| Lean proof | theorem bytes, tactic attempts, per-attempt timeout, backend restarts, generated auxiliary lemmas. |
| Persistent data | cache entries, companion bytes, age/generation policy. |

Limits govern both accepted shape and cumulative work. A one-million-node lazy input should not be traversed before a count cap is noticed. The current codebase already pays close attention to productive admission; new modules should follow the same discipline.

## 12.3 Budget ownership

Budgets should be hierarchically scoped:

- a command budget caps all behavioral work;
- each engine/sketch/CHC lane receives a fair sub-budget;
- each candidate has an interpretation and query budget;
- one live worker scope owns an absolute deadline and cooperative checkpoints;
- proof generation has a separate budget so it cannot consume counterexample discovery time.

Unused sub-budgets may be returned to the scheduler under a deterministic rule. Dynamic reallocation decisions enter observability but need not enter semantic fingerprints because they affect completeness/performance, not the meaning of evidence. They do enter a search-run identity for reproducibility.

## 12.4 One process or several

The current Length design deliberately uses a scoped, capability-probed worker and serial query lease. That is the correct starting point. For richer synthesis:

**Single process, multiple protocol scopes.**
Lowest overhead; ordinary solver, sketch and Optimize work can share a process only if each scope resets to a capability-checked baseline and protocol identities prevent cross-mode misuse.

**Separate process per protocol family.**
Stronger isolation and simpler state machines; recommended for fixedpoint/Spacer and perhaps Optimize initially.

**Portfolio processes.**
Run several Z3 parameter profiles or theory encodings in parallel. Useful later, but each observation carries its exact process/profile identity and only replayed evidence can win a race.

One observation made while checking chapter B argues for the second option more strongly than isolation alone. Z3 module parameters set with `set-option` are process-global, not context-local: after `(set-option :smt.core.minimize true)` has been issued once, as the core-producing scope in listing B.4 does, the fixedpoint query of listing B.6 answers `unknown` in a fresh context of the same Z3 5.0.0 process, whereas in an untouched process it answers `sat` with the expected invariant. A single shared process therefore needs not only per-scope resets but a proof that no protocol family's parameters can reach another's, which is exactly the cross-mode misuse the protocol identities are meant to exclude. Until such a proof exists, one process per protocol family is the safer default.

A process crash or protocol mismatch invalidates the worker, not the verified candidates or counterexample inputs already independently replayed.

## 12.5 Determinism

Full solver determinism cannot be assumed, but user-visible ordering can be made stable:

- canonicalize contract clauses, symbols and grammar alternatives under defined source-order rules;
- use deterministic fresh-name allocation;
- pin Z3 version and parameters in reproducibility mode;
- use one query at a time per worker;
- simplify counterexamples under a deterministic domain order;
- replay and insert counterexamples in source/query order;
- sort or stable-partition candidates by explicit keys, never by model-print order;
- use iterative SAT for primary size optimization when exact reproducibility matters;
- record the solver seed and all tactic parameters in execution identity.

A different but valid counterexample should not alter correctness, but it may alter later search. Transcripts should record enough identity and the decoded sample to reproduce the run under the same setup.

## 12.6 Counterexample minimization

Small counterexamples improve both diagnostics and future replay. Use a domain-owned sequence:

1. normalize the raw decoded input;
2. try existing pure query-owned simplification;
3. lexicographically reduce naturals, lengths, tags or atom counts by additional solver queries;
4. remove irrelevant function-table entries or state distinctions;

5. replay every proposed simplification against the exact problem;
6. stop at explicit query/work bounds.

The original replayed input remains the fallback. A simplification is not accepted because it satisfies a transformed formula; it must reproduce the violation under the original checked problem.

## 12.7 Progressive deepening

Sketch and recursive search should widen one dimension at a time under a deterministic schedule:

1. grammar depth/node count;
2. provider inventory tier;
3. number of case splits;
4. literal magnitude or coefficient range;
5. sample bank size considered in the solver;
6. recursive template depth or invariant grammar;
7. theory profile from QF-LIA to richer sequence/array or CHC encodings.

Each round has its own grammar/query fingerprint. A no-sketch result is reported with the exact bounds. Learned samples can cross rounds when the domain session and contract match; grammar nogoods generally cannot cross grammar changes unless the new grammar explicitly embeds the old choice identities.

## 12.8 Reproducibility manifest

A transcript or saved result should be able to emit a manifest containing:

- Leant, Djex and Z3 revisions/versions;
- Lean toolchain, project and backend identity;
- contract and configuration fingerprints;
- target text and normalized target fingerprint;
- engine and provider-inventory policies;
- grammar and progressive-bound schedule;
- solver executable identity, argv, parameters and seed;
- counterexample sequence;
- accepted candidate exact text and origin fingerprint;
- proof theorem source and result.

This manifest is diagnostic, not authority. The actual receipts remain opaque in-process values or snapshot companion data.

Chapter 13

# Diagnostics and failure taxonomy

## 13.1 Why structured failures matter

The current Length handoff enumerates precise fail-closed phases rather than flattening them into "no semantic data." That approach should be retained. Behavioral synthesis has enough stages that a string-only error would be nearly impossible to debug.

Failures should be grouped by ownership:

1. source/contract admission;
2. target correspondence;
3. verified-origin recovery;
4. domain session and provider-law sealing;
5. candidate graph interpretation;
6. query construction;
7. process/protocol/response;
8. model decoding and replay;
9. sketch materialization and Lean verification;
10. proof-obligation rendering and Lean proof;
11. cache/association mismatch;
12. resource and cancellation.

## 13.2 Representative refusal vocabulary

```
data BehaviorPreparationRefusal
  = BehaviorNotTypedRoute CandidateRenderingRoute
  | BehaviorMissingExactOrigin
  | BehaviorRetargetedSource
  | BehaviorUnexpectedPremises Natural
  | BehaviorRequestContextUnsupported Natural
  | BehaviorRendererCorrespondenceLost
  | BehaviorTargetRoleMismatch TargetRoleError
```

```
  | BehaviorFamilyUnavailable SourceName
  | BehaviorProviderUnavailable SourceName
  | BehaviorProviderLawMismatch SourceName
  | BehaviorGraphUnavailable TypedGraphAbsence
  | BehaviorDomainUnsupportedConstruct DomainConstruct
  | BehaviorResidualConstraint ConstraintSummary
  | BehaviorSessionRejected DomainSessionError
  | BehaviorContractRejected DomainContractError
  | BehaviorProblemRejected DomainProblemError

data BehaviorLiveFailure
  = BehaviorQueryRejected QueryError
  | BehaviorWorkerUnavailable WorkerError
  | BehaviorProtocolRejected ProtocolError
  | BehaviorResponseRejected ResponseError
  | BehaviorModelRejected ModelError
  | BehaviorReplayRejected ReplayError
  | BehaviorAssociationMismatch ReplayMismatch
  | BehaviorDeadlineExceeded DeadlinePhase
  | BehaviorCancelled
```

Listing 13.1: Illustrative generic refusal classes.

Public diagnostics may sanitize source constraints or OS details, while debug reports can retain bounded structured context. No failure should include unbounded solver output or private process handles.

## 13.3 Clause and grammar source spans

Every contract clause and sketch production should have a stable source identity and optional source span. Z3 assumption literals map back to these identities. An unsat core or counterexample can therefore point to:

- the exact contract clause;
- the target role or provider law involved;
- the selected grammar production;
- the candidate graph occurrence;
- the Lean theorem hypothesis generated from it.

Source spans are diagnostic only. Semantic identities use normalized checked structure so reformatting a JSON file does not invalidate a contract unless ordering is semantically significant.

## 13.4 Unknown reasons

Unknown should be a structured sum, not one status string:

- solver returned `unknown` with bounded reason;
- deadline or cancellation;
- response artifact exceeded limits;
- unsupported theory/profile combination;
- recursive unfolding incomplete;

- proof tactic exhausted;
- candidate semantics over-approximate only;
- provider behavior unavailable;
- finite sample coverage only;
- sketch grammar exhausted but ordinary search incomplete.

This lets policies distinguish “preserve on timeout” from “require proof and reject unproved” without pretending the underlying statuses are the same.

Chapter 14

# Testing and evaluation

## 14.1 Testing philosophy

The main risk is not that Z3 returns an obviously wrong status. It is that an identity, role, source name, graph occurrence, provider law, model value, approximation direction or proof proposition is associated with the wrong artifact. Tests must therefore emphasize boundaries and adversarial reassociation.

## 14.2 Unit and property tests

Every new module should have tests for:

- productive admission at each limit and first-excess behavior;
- canonical normalization and alpha-renaming invariance;
- fingerprint sensitivity to every semantic field and insensitivity to raw allocation coordinates;
- target-role arity/order correspondence;
- source-name resolution and ambiguous/missing-provider failures;
- graph/schema resealing and dead-node audits;
- model decoder sorting, range, duplicate and missing-value checks;
- replay associativity: foreign problem/query/candidate receipts fail;
- exact versus over-/under-approximation polarity;
- sample-bank deduplication and LRU order;
- sketch materialization, certificate preservation and exact-model blocking;
- core-to-nogood translation;
- Lean and SMT renderer differential evaluation on bounded inputs;
- timeout, cancellation, EOF and cumulative-output precedence.

QuickCheck-style generators should produce small checked inventories, contracts, term graphs, sketches and model values. Shrinkers must preserve well-formedness so failures reduce to meaningful boundary cases.

## 14.3 Differential tests

Use several independent oracles where possible:

- domain evaluator versus generated Lean `#eval` on finite concrete types;
- native sequence encoding versus array-plus-length encoding on bounded instances;
- finite exhaustive validator versus Z3 query/replay;
- ordinary Exference candidate versus the same candidate materialized from a singleton sketch;
- iterative SAT optimization versus native Optimize on small grammars;
- recursive bounded evaluator versus CHC counterexample traces;
- Z3 5.0.0 versus a pinned nearby version for protocol and canonical model-decoder tests, without assuming identical models.

Disagreement should never be resolved by choosing the more favorable result. It is a diagnostic failure until the domain explains the exactness difference.

## 14.4 Golden transcript tests

Leant should add golden tests for:

- each behavior mode and no-option compatibility;
- concrete, finite-abstract, relational and CHC counterexamples;
- provider-assumption presentation;
- unknown, inapplicable, bounded and proof-grade statuses;
- sketch progress and learned-nogood counts under pinned profiles;
- backend restart during proof mode;
- snapshot/pickle restoration of contract and bank companions;
- exact preservation of existing Length ranking output when new modes are unused.

Golden text should not contain nondeterministic raw model names. The domain renderer canonicalizes them first.

## 14.5 Adversarial identity tests

Explicitly attempt to mix:

- same rendered term from two candidate origins;
- same graph with and without certificate carrier;
- same provider name from two inventories;
- same contract under different spine/sequence family identities;
- model values from another query with identical symbol spelling;
- a counterexample input from a different provider-law basis;

- a sketch model from a grammar with reordered alternatives;
- a CHC invariant from another recursive schema;
- a Lean proof receipt after environment reset or shadowing.

Every case must fail closed at the earliest documented boundary.

## 14.6 Protocol fuzzing

The robust current SMT-LIB parser/causal stack is a strong base. Extend fuzzing for:

- arbitrary chunk boundaries and legal whitespace;
- delayed barriers and extra responses;
- malformed numerals, bit-vector literals, arrays, datatypes and models;
- huge but shallow and deep but small S-expressions;
- solver errors in every phase;
- cores with unknown/duplicate assumptions;
- Optimize objective responses in unexpected order;
- fixedpoint answers and invariant artifacts exceeding bounds;
- cancellation between write, frame completion, model decode and replay.

Use the real Z3 executable in integration tests and pure fake transports for exhaustive phase-machine tests.

## 14.7 Benchmark corpus

A useful benchmark suite should contain positive, negative and near-miss candidates. table 14.1 proposes initial groups.

Table 14.1: Proposed behavioral-synthesis benchmark groups.

| Group | Targets | Defects to distinguish |
|---|---|---|
| Length/shape | identity, append, reverse, duplicate, drop, flatten, tree mirror | constant output, lost branch, duplicated subtree, wrong affine transfer. |
| Constructor behavior | Option/Except bind/map/default, sum eliminators | constant constructor, wrong branch, ignored callback, payload mismatch. |
| State wiring | reader/state/writer bind, lens laws | initial versus intermediate state, wrong projection, duplicated call, lost log. |
| Sequence | reverse, map, append, take/drop, rotation | correct size but wrong order/index/source element. |
| Bag/permutation | permutation, partition, dedup, insertion | loss, duplication, wrong side, unstable order. |
| Scalar | clamp, abs, min/max, affine transforms | boundary errors, sign mistakes, off-by-one, wrong branch guard. |
| Bit-vector | saturating add, masks, byte swap, rotates | overflow, signedness, shift-width, mask constant errors. |

| Group | Targets | Defects to distinguish |
|---|---|---|
| Recursive | folds, reverse accumulator, tree size/map | insufficient invariant, wrong recursive argument, nontermination schema. |
| Higher-order | composition, map/fold callbacks, state callbacks | argument permutation, callback ignored/reused, wrong instantiation. |

For each target, store:

- target type and environment;
- contract and provider laws;
- known correct candidates;
- minimally different incorrect candidates;
- expected smallest counterexamples;
- expected proof strategy;
- grammar bounds required for typed-sketch discovery;
- performance baselines and solver profile.

## 14.8 Metrics

Measure more than wall-clock time:

- candidates generated, Lean-verified, bank-rejected and solver-queried;
- counterexamples discovered, replay hits and minimization work;
- semantic-equivalence classes under the bank;
- sketch variables, clauses, models, blocks, cores and learned nogoods;
- query and response bytes;
- solver and proof time separately;
- fraction of positive results proved in Lean;
- inapplicable/unknown rates by domain and source construct;
- regression against unconstrained synthesis ordering and completeness;
- reproducibility across repeated pinned runs.

The primary product metric is not "number of Z3 calls." It is how many type-correct but behaviorally wrong candidates are eliminated before expensive search, and how often a final candidate carries a Lean proof.

Chapter 15

# Implementation roadmap

## 15.1 Roadmap principles

The roadmap is organized as vertical slices. Each milestone ends with an end-to-end user-visible capability and preserves the default no-contract path. Large horizontal refactors are deferred until a working slice proves the abstraction.

## 15.2 Milestone 0: compatibility and measurement harness

**Purpose:** establish a nonregression base before behavior can remove candidates.

**Changes:**

- snapshot current `:synth` and Length ranking transcripts at the inspected revisions;
- add observability for handoff refusals, replay hits, pure validators, live queries and candidate rank movement;
- add stable manifest output with Leant/Djex/Z3 identities;
- create a compact benchmark corpus for length-preserving and state-threading examples;
- pin a supported Z3 5.0.0 profile while retaining configured executable identity checks.

**Acceptance criteria:**

- no-option and existing Length configurations are byte-for-byte unchanged except for explicitly enabled debug metrics;
- every existing whole-tree test passes;
- repeated pinned runs produce identical candidate order and normalized counterexample presentation for the selected corpus.

## 15.3 Milestone 1: hard filtering over current Length

**Purpose:** prove the new selection authority with no new semantic domain.

**Changes:**

- implement nominal behavioral-selection input and sealed decisions;
- add `--behavior-mode filter`;

- adapt the current scalar and spine-pair Length assessment to produce rejection decisions after replayed counterexamples;
- preserve on `unsat`, `unknown`, inapplicability and assessment failure by default;
- expose rejected candidates and exact counterexamples in diagnostics.

**Acceptance criteria:**

- a known length-changing verified candidate is absent from the selected result only when a candidate-associated replay receipt exists;
- foreign/reordered/duplicated decision handles fail the selection seal;
- a Z3 `sat` model which fails replay cannot reject a candidate;
- disabling filter mode restores exact existing ranking behavior.

## 15.4 Milestone 2: command-local candidate CEGIS

**Purpose:** reuse learned counterexamples across ordinary candidates and engine batches.

**Changes:**

- introduce a command-scoped counterexample bank for Length inputs;
- replay the bank before all pure/live stages;
- retain bank statistics and deterministic priority;
- continue Exference search or progressive engine batches after rejection;
- add semantic signatures over bank results for optional duplicate suppression.

**Acceptance criteria:**

- one solver-discovered length counterexample rejects later candidates without another solver call;
- every later rejection is freshly replayed and candidate-associated;
- bank identity changes when the contract, target, session, provider basis or model profile changes;
- bank limits and first-excess behavior are productive and tested with cyclic/lazy inputs where relevant.

## 15.5 Milestone 3: generic behavioral orchestration

**Purpose:** extract shared control structure while preserving current Length bytes.

**Changes:**

- add first-class behavioral-domain packages and outer contract/configuration envelopes;
- implement a Size/Shape-v1 adapter over current Length scalar/pair paths;
- move selection, bank, unknown policy and presentation to generic modules;
- keep existing Length file decoders and entry points as compatibility wrappers;
- add `:behavior-check` and domain inspection.

**Acceptance criteria:**

- all existing Length fingerprints, file schemas, refusals and ranking transcripts remain unchanged;
- the same checked Length problem can be reached through the compatibility and generic adapters and has identical identity;
- an intentionally foreign domain value cannot be combined through representation coercion or existential unpacking.

## 15.6 Milestone 4: constructor behavior and state provenance

**Purpose:** deliver behavior beyond length before tackling general sequences.

**Changes:**

- add finite constructor-tag and payload-provenance semantics for exact checked datatypes;
- add uninterpreted callback/state terms and tuple projections;
- implement contracts for `Option.bind` and state bind;
- extend the typed term interpreter only with exact graph forms needed by these contracts;
- add relational model rendering and replay.

**Acceptance criteria:**

- always-none, wrong-branch and wrong-state-threading candidates receive small replayed countermodels;
- correct direct candidates survive without provider assumptions;
- opaque callback/state values cannot be inspected outside admitted operations;
- total/pure uninterpreted-function assumptions are displayed in every relational receipt.

## 15.7 Milestone 5: nonrecursive typed sketches

**Purpose:** make Z3 select terms from a Djex-checked finite grammar.

**Changes:**

- define checked selector-DAG grammar and canonical grammar fingerprint;
- generate small normal-form grammars for constructor/state examples;
- implement incremental sketch protocol, model decoder, materializer and exact blocking;
- preserve provider/constructor origins and certificate associations;
- add a fair sketch lane beside ordinary Djinn/Exference lanes.

**Acceptance criteria:**

- singleton grammars materialize byte/graph-equivalent candidates to ordinary construction where expected;
- Z3 synthesizes `Option.bind` and correct state bind within declared bounds;
- every materialized candidate is Lean-verified before behavioral checking;
- model/grammar reassociation, alternative reordering and invalid literal decoding fail closed;
- “no sketch” is labeled with exact grammar bounds and never as type uninhabitation.

## 15.8 Milestone 6: proof-grade Size/Shape and branch/state contracts

**Purpose:** close the positive authority loop.

**Changes:**

- generate Lean candidate definitions and theorem propositions;
- link known provider laws to checked Lean theorems;
- implement bounded tactic portfolios for definitional equality, cases, simp and omega;
- retain proof receipts and present assumptions explicitly;
- add `--behavior-mode prove`.

**Acceptance criteria:**

- direct Option/state candidates produce Lean-accepted theorems;
- a simple constructor/provider length candidate produces a Lean-accepted theorem under checked library laws;
- a solver-unsat result without a theorem is not labeled proved;
- proof timeout and theorem refutation have distinct statuses;
- environment reset invalidates or replays proof receipts correctly.

## 15.9 Milestone 7: sequence/index and bag/permutation domains

**Purpose:** distinguish order and multiplicity.

**Changes:**

- implement native-sequence and array-plus-length profiles;
- add guarded index counterexamples and finite-atom sequence replay;
- add symbolic-witness and finite-atom bag count profiles;
- layer combined permutation and partition contracts;
- add provider laws/theorems for append, reverse, map, filter and partition as available.

**Acceptance criteria:**

- identity and reverse are separated by an index counterexample;
- equal-length duplicate/delete candidates are rejected by bag witnesses;
- out-of-bounds sequence reads are impossible in checked queries and replay;
- native-sequence and array profiles agree on bounded differential tests;
- combined receipts identify whether size, bag or order failed.

## 15.10 Milestone 8: scalar templates and bit-vector synthesis

**Purpose:** exploit Z3's strongest decidable theories for generative synthesis.

**Changes:**

- add coefficient/threshold/literal parameter slots;
- add iterative size/constant optimization and optional native Optimize;
- implement width-indexed bit-vector grammar and exact model decoding;
- generate Lean arithmetic/bit-vector obligations using `omega` and `bv_decide`;
- benchmark saturating arithmetic, masks, byte permutations and clamps.

**Acceptance criteria:**

- selected constants and masks round-trip exactly into Lean syntax;
- signedness/width/profile mismatches fail before materialization;
- proof-grade finite bit-vector candidates are accepted by Lean;
- iterative SAT and Optimize agree on small benchmark optima under pinned objective order.

## 15.11 Milestone 9: sound prefix pruning

**Purpose:** move behavioral reasoning into Exference's partial search.

**Changes:**

- define over-approximate completion summaries for Size/Shape, tags, dependencies and selected scalar templates;
- add assumption literals for search decisions;
- seal pruning receipts and core-derived nogoods;
- integrate with queue pruning and completion-status reporting;
- retain an audit that every semantic prune had a receipt.

**Acceptance criteria:**

- every pruned prefix is independently covered by the sealed over-approximation and exact bank/contract identity;
- replacing the summary by a deliberately under-approximate test implementation is rejected by type/API tests;
- exhaustive finite-grammar results remain correct with pruning enabled/disabled;
- search reports distinguish semantic pruning from ordinary resource pruning.

## 15.12 Milestone 10: recursive CHC path

**Purpose:** synthesize and prove selected recursive templates.

**Changes:**

- implement checked relation/rule IR and separate fixedpoint worker profile;
- lower structurally recursive templates to Horn clauses;
- decode concrete traces and bounded invariant candidates;
- translate admitted invariants into Lean lemmas;

- combine with Lean termination checking and induction proof generation.

**Acceptance criteria:**

- wrong recursive-argument candidates yield replayable abstract traces;
- a length-preserving accumulator example yields an invariant that Lean proves;
- Spacer safety without a Lean proof is labeled solver-only;
- recursive SMT unfolding is never mistaken for induction;
- fixedpoint protocol artifacts cannot enter ordinary solver or Optimize replay paths.

## 15.13 Recommended first deliverable

The first practically compelling deliverable combines Milestones 1, 2, 4 and a narrow slice of 5:

- hard filter and command-local CEGIS;
- existing Length contracts;
- new Option/state relational contracts;
- small selector-DAG templates for case and tuple plumbing;
- Lean verification of every candidate;
- clear replayed counterexamples;
- no positive proof claim beyond current finite validators.

This demonstrates the entire feedback architecture and solves examples that type-directed search alone cannot rank reliably, while avoiding sequence quantifiers, bag semantics and recursion.

Chapter 16

# Risks, tradeoffs, and rejected alternatives

## 16.1 Risk matrix

Table 16.1: Principal risks and mitigations.

| Risk | Failure mode | Mitigation |
|---|---|---|
| Semantic mismatch | SMT abstraction proves/refutes a property different from the intended Lean law. | Narrow versioned domains, explicit exactness, differential evaluation, Lean theorem generation, fail-closed unsupported constructs. |
| Identity drift | Model/evidence/provider/graph is paired with another candidate or environment. | Nominal opaque values, exact fingerprints, occurrence-sealed handoff, adversarial reassociation tests. |
| Solver over-trust | `unsat` or a printed invariant is presented as proof. | Observation/evidence/proof taxonomy; default counterexample use; Lean proof receipts for positive authority. |
| Search starvation | Sketch or behavioral checks consume all budget and suppress existing engines. | Fair lanes, independent budgets, protected Djinn baseline/refutation behavior, progressive deepening. |
| Performance blowup | Samples times candidates or rich theories become intractable. | Replay-first staging, bank prioritization, signatures, query simplification, strict limits, domain-specific profiles. |
| Grammar incompleteness | "No sketch" is misread as no program. | Grammar fingerprint/bounds in status; ordinary search remains; no general negative claim. |
| Provider-law unsoundness | Assumed transfer hides incorrect provider behavior or dictionaries. | Explicit trust classes, occurrence-specific authority, ground discharge, Lean theorem linking, assumptions in output. |
| Higher-order model mismatch | Uninterpreted functions assume total purity unlike source behavior. | Explicit relational model label; restrict to wiring/extensional laws; Lean proof required for source theorem. |
| Recursive unsoundness | Bounded unfolding or CHC abstraction omits behavior. | Separate exactness profile, trace replay, invariant translation, Lean induction and termination proof. |

| Risk | Failure mode | Mitigation |
|---|---|---|
| Solver/version drift | Models, optima or tactic results change across releases. | Pin/profile identity, deterministic canonicalization, iterative SAT fallback, replay, manifests. |
| API overgeneralization | One abstract interface obscures domain-specific invariants and weakens types. | First-class closed dictionaries over nominal domain types; shared orchestration only; Length adapter before refactor. |
| UX overload | Users cannot tell proof from heuristic success. | Compact strength labels, detailed explanation command, assumptions and bounds shown, modes explicit. |

## 16.2 Rejected alternative: translate arbitrary Lean expressions to SMT

A broad Lean-to-SMT translator would appear to maximize reuse, but it is the wrong foundation here. It would need semantics for dependent pattern matching, universes, proof arguments, type classes, quotient and subtype coercions, partiality, recursion, effects, opaque constants and evaluation strategy. Any practical version would silently model only a fragment, recreating the need for exact domain descriptors and source correspondence while losing the current typed graph and provider certificate architecture.

Narrow behavioral interpreters are less glamorous but more honest and testable. They can expand one source construct at a time under exact schemas.

## 16.3 Rejected alternative: use Z3 as the primary type-directed synthesizer

Z3 can encode simple typing rules over AST datatypes, and its official datatype guide even demonstrates type constraints for a simply typed lambda calculus. But Djex already has mature proof/search engines with rank-N handling, provider evidence, nominal source types and Lean-specific rendering. Reimplementing those rules in SMT would:

- duplicate and likely weaken the existing type boundary;
- make binders and alpha-equivalence expensive;
- complicate polymorphic provider instantiation;
- produce terms detached from exact source occurrence evidence;
- move debugging from Haskell invariants into opaque solver models.

Typed selector sketches capture the useful finite-choice part without duplicating typing.

## 16.4 Rejected alternative: trust unsat for pruning everywhere

An unsat answer is only as sound as the exact query. For complete candidate counterexample queries in a trusted-solver mode, it may be reasonable to accept external-solver authority. For partial terms, however, pruning additionally depends on a sound completion abstraction. Applying unsat pruning to an under-approximation or incomplete provider model can delete valid programs. The default architecture therefore uses unsat for:

- finite sketch infeasibility within explicit grammar bounds;

- nogood learning only under checked over-approximations;
- proof/invariant guidance;
- positive ranking signals;
- optional explicitly trusted external-solver mode.

Lean proof mode is the preferred route to unconditional positive claims.

## 16.5 Rejected alternative: one monolithic contract theory

A single expression language containing arithmetic, sequences, bags, states, costs, bit-vectors, ADTs and recursion would quickly acquire ambiguous combinations. What does bag equality mean for an infinite sequence? Does a cost expression count graph nodes or runtime reductions? Is an uninterpreted callback pure? Does bit-vector addition coerce to natural length? Domain composition should be explicit, with a checked product of observation algebras and a declared lowering profile. Shared syntax does not imply shared semantics.

## 16.6 Tradeoff: subprocess versus FFI

A direct C API binding could reduce parsing overhead and expose rich ASTs, but the existing subprocess architecture has important advantages:

- process crashes and memory corruption are isolated;
- executable identity and pinning are explicit;
- cancellation/deadline/cleanup ownership is already strong;
- the SMT-LIB transcript is reproducible and inspectable;
- no Haskell FFI lifetime/refcounting layer enters the trusted boundary.

The proposal recommends retaining subprocess execution through the first sketch, Optimize and CHC milestones. Reconsider FFI only after profiling shows parsing/transport dominates, and preserve the same typed plan, replay and observation semantics above either transport.

## 16.7 Tradeoff: generic type class versus closed dictionary

A type class gives elegant static dispatch inside Djex. A first-class existential dictionary gives controlled runtime domain selection and clearer versioned registration. The two can coexist: implement each domain through a private type class, then package the exact methods in a closed dictionary exposed to orchestration. Avoid an open global registry in the stable API until plugin/version policy is designed.

## 16.8 Open research questions

Several questions deserve experiments rather than premature commitments:

- How much of Exference's search frontier can be converted into compact selector DAGs without losing its ranking advantages?
- Which sample-selection strategy gives the best reduction: smallest counterexample, most recent, maximum class split, or solver-derived diversity?

- Can carrier-aware typed graph fingerprints support useful beta/eta-normal semantic deduplication without becoming authority?
- Which sequence profile is faster and more predictable for common contracts under Z3 5.0.0?
- Can Lean's existing simplifier and arithmetic tactics prove the current Length interpretation equations at useful scale?
- How often can Spacer invariants be translated into human-sized Lean induction lemmas automatically?
- Can provider theorem discovery reuse Leant's live inventory and active-instance provenance without exploding search?
- Is Synthesiz3 competitive with selector-DAG sketches on first-order template benchmarks?
- Which completion abstractions are strong enough to prune Exference while remaining easy to prove sound by construction?
- Can a finite-model finder over abstract atoms produce better polymorphic counterexamples than sequence/array models alone?

Chapter 17

# Conclusion

LEANT and DJEX are now unusually well positioned for behaviorally constrained synthesis. The difficult prerequisites are already present: a shared parser-independent synthesis foundation; terminating and heuristic search with honest completion statuses; exact Lean-side source provenance; verified candidate occurrences; typed term graphs and certificate associations; checked inventories and provider resolution; collision-free canonical identities; a generic behavioral problem envelope; a narrow but sophisticated symbolic interpreter; a bounded canonical SMT lowering; and a robust live Z3 process/replay stack.

The next step should not be another isolated semantic ranker and should not be a wholesale SMT encoding of Lean. It should be a feedback architecture that uses those foundations at progressively earlier points:

1. replayed counterexamples reject wrong verified candidates;
2. command-local banks turn ordinary enumeration into CEGIS;
3. Djex-prepared typed sketches let Z3 choose finite constructions and parameters;
4. sound completion summaries prune impossible prefixes;
5. Optimize chooses among hard-correct sketches;
6. Spacer proposes invariants for recursive relational semantics;
7. Lean accepts the final term and, when requested, the final behavioral theorem.

The central technical discipline is to keep four things distinct:

- *the candidate* and its exact source/typed origin;
- *the behavioral model* and its explicit exactness and provider assumptions;
- *the solver observation* and its bounded execution identity;
- *the authority* obtained by replay, exhaustive validation, or Lean proof.

With that discipline, Z3 becomes more than a post-hoc checker without becoming an unexamined oracle. It can search the behavioral space, generate discriminating counterexamples, solve finite typed choices, learn impossible combinations, optimize implementations and discover recursive invariants. Djex retains control of program structure and source meaning. Lean remains the final arbiter of executable syntax and theorem correctness. This division is both technically realistic and capable of producing synthesis results substantially more precise than types alone.

Appendix A

# Extended Haskell interface sketch

This appendix collects a more complete conceptual interface. It is not intended to compile unchanged; it makes ownership and nominal association explicit enough to guide implementation reviews.

## A.1 Domain package

```haskell
data DomainCapabilities = DomainCapabilities
  { supportsCandidateReplay       :: Bool
  , supportsPureValidation        :: Bool
  , supportsUniversalSolverQuery  :: Bool
  , supportsTypedSketches         :: Bool
  , supportsCompletionSummaries   :: Bool
  , supportsOptimization          :: Bool
  , supportsFixedpoint            :: Bool
  , supportsLeanObligations       :: Bool
  }

data SomeBehavioralDomain source verified =
  forall d. SomeBehavioralDomain
    { someDomainIdentity       :: BehavioralDomainIdentity d
    , someDomainCapabilities   :: DomainCapabilities
    , someAdmitSource          :: DomainSourceLimits d
                               -> source
                               -> Either (DomainSourceError d)
                                         (AdmittedSource d)
    , someSealSession          :: Inventory
                               -> AdmittedSource d
                               -> Either (DomainSessionError d)
                                         (CheckedDomainSession d)
    , someSealContract         :: CheckedDomainSession d
                               -> AdmittedSource d
                               -> Either (DomainContractError d)
                                         (CheckedDomainContract d)
    , someSealVerifiedProblem  :: CheckedDomainSession d
                               -> CheckedDomainContract d
                               -> verified
                               -> Either (DomainProblemError d)
                                         (CheckedDomainProblem d)
    , someReplayInput          :: CheckedDomainProblem d
                               -> DomainInput d
```

```
                              -> Either (DomainReplayError d)
                                        (DomainReplayResult d)
    , someBuildCounterexample  :: DomainQueryLimits d
                              -> CheckedDomainProblem d
                              -> Either (DomainQueryError d)
                                        (CheckedCounterexampleQuery d)
    , someDecodeCounterexample :: CheckedCounterexampleQuery d
                              -> AssociatedRawModel d
                              -> Either (DomainModelError d)
                                        (DomainInput d)
    , someBuildProof           :: CheckedDomainProblem d
                              -> Either (DomainProofError d)
                                        (LeanProofObligation d)
    }
```

Listing A.1: Extended domain package.

The session and contract are separate because a session may be reused across command-local contracts if the domain chooses, while the checked problem always binds one exact contract. The concrete Length adapter may retain its present atomic session/contract invariants internally.

## A.2 CEGIS session

```
data CegisSession d candidate = CegisSession
  { cegisDomainSession       :: CheckedDomainSession d
  , cegisContract            :: CheckedDomainContract d
  , cegisBank                :: CounterexampleBank d
  , cegisCandidateSeen       :: FingerprintSet CandidateFingerprintSubject
  , cegisSemanticClasses     :: SemanticClassTable d candidate
  , cegisLimits              :: CegisLimits
  , cegisStatistics          :: CegisStatistics
  , cegisSearchRunIdentity   :: SearchRunFingerprint
  }

data CegisStep d candidate
  = CegisCandidateRejected (CounterexampleEvidence d)
  | CegisCandidateSurvives  (PositiveObservation d)
  | CegisCandidateProved    LeanProofReceipt
  | CegisCandidateUnknown   BehavioralUnknown
  | CegisBankExtended       (CounterexampleInput d)
  | CegisExhausted          SearchCompletion
```

Listing A.2: Candidate-level CEGIS state.

A CEGIS session owns no raw engine continuation directly. Leant's scheduler owns engine batches and feeds verified receipts into the domain session. This keeps the Djinn/Exference completion vocabulary unchanged.

## A.3 Selection seal

```
data SelectionCandidate epoch candidate = SelectionCandidate
  { selectionOriginalIndex :: Natural
  , selectionVerified      :: Verified candidate
  }
```

```
type role SelectionCandidate nominal nominal

data SelectionDecision epoch candidate
  = KeepCandidate    (SelectionCandidate epoch candidate)
                     BehavioralKeepReason
  | RejectCandidate  (SelectionCandidate epoch candidate)
                     AssociatedRejectionEvidence
  | MarkUnknown      (SelectionCandidate epoch candidate)
                     BehavioralUnknown
  | MarkInapplicable (SelectionCandidate epoch candidate)
                     DomainRefusal

data BehavioralSelectionBatch candidate -- constructor private

sealBehavioralSelection
  :: SelectionLimits
  -> UnknownPolicy
  -> SelectionInput epoch candidate
  -> [SelectionDecision epoch candidate]
  -> Either SelectionSealError (BehavioralSelectionBatch candidate)
```

Listing A.3: Nominal selection seal in more detail.

The sealer checks:

1. candidate input admission before decision admission;
2. exactly one decision per original occurrence;
3. no duplicate or out-of-range index;
4. evidence candidate fingerprint and origin association;
5. decision class permitted by policy;
6. selected output order, ordinarily the stable input order unless an additional validated ranking is composed.

## A.4 Sketch grammar

```
data CheckedSketchGrammar d local ty = CheckedSketchGrammar
  { sketchGrammarDomain       :: BehavioralDomainIdentity d
  , sketchGrammarInventory    :: Fingerprint InventoryFingerprintSubject
  , sketchGrammarTarget       :: ty
  , sketchGrammarLocals       :: Vector (CheckedLocal local ty)
  , sketchGrammarNodes        :: Vector (CheckedSketchNode d local ty)
  , sketchGrammarRoot         :: SketchNodeId
  , sketchGrammarFingerprint  :: Fingerprint SketchGrammarSubject
  , sketchGrammarLimits       :: SketchGrammarLimits
  }

data CheckedSketchNode d local ty = CheckedSketchNode
  { checkedSketchNodeType         :: ty
  , checkedSketchNodeAlternatives :: NonEmpty
      (CheckedSketchAlternative d local ty)
  }
```

```haskell
data CheckedSketchAlternative d local ty
  = CheckedLocalUse local
  | CheckedPreparedProvider (PreparedProviderOccurrence local ty)
  | CheckedApplication SketchNodeId SketchNodeId
  | CheckedLambda (CheckedBinder local ty) SketchNodeId
  | CheckedConstructor (CheckedConstructorOccurrence ty) [SketchNodeId]
  | CheckedCase SketchNodeId [CheckedSketchBranch d local ty]
  | CheckedDomainPrimitive (DomainPrimitive d) [SketchNodeId]
  | CheckedParameter (DomainParameterSlot d)
```

Listing A.4: Typed sketch grammar with prepared occurrences.

Constructors are private. Grammar construction checks type equality, binder scope, node topological order, provider certificates and domain primitive arity. A materialized graph is freshly sealed under the same shared type structure rather than trusting construction alone.

## A.5 Approximation and pruning

```haskell
data CompletionPrefix grammar -- exact partial grammar/search state

data CompletionOverApproximation d grammar =
  CompletionOverApproximation
    (BehavioralProblem PrefixProblemSubject)
    (OverApproximationProof d grammar)
    (DomainSummary d)

data CheckedPruningQuery d grammar

data PruningEvidence d grammar -- constructor private

buildCompletionOverApproximation
  :: CompletionSummaryLimits
  -> CheckedDomainSession d
  -> CheckedDomainContract d
  -> CheckedSketchGrammar d local ty
  -> CompletionPrefix grammar
  -> Either (CompletionSummaryError d)
            (CompletionOverApproximation d grammar)

replayPruningUnsat
  :: CompletionOverApproximation d grammar
  -> AssociatedUnsatObservation
  -> Either ReplayMismatch (PruningEvidence d grammar)
```

Listing A.5: Completion-summary interface.

The `OverApproximationProof` is a domain-owned opaque receipt that the summary constructor followed a checked compositional rule. It need not be a formal theorem initially, but it must be unforgeable and exhaustively tested. A later Lean formalization can strengthen it.

## A.6 Proof obligations

```haskell
data LeanProofObligation d = LeanProofObligation
```

```
  { proofProblemFingerprint  :: Fingerprint ProblemFingerprintSubject
  , proofCandidateDefinition :: LeanDeclarationSource
  , proofTheoremDeclaration  :: LeanDeclarationSource
  , proofRequiredImports     :: [LeanModuleName]
  , proofProviderTheorems    :: [CheckedProviderTheorem]
  , proofSuggestedPortfolio  :: NonEmpty LeanProofStrategy
  , proofExactness           :: ProofExactness d
  }

data ProofExactness d
  = ProofOfSourceContract
  | ProofConditionalOn [LeanHypothesisIdentity]
  | ProofOfBoundedContract DomainBound
  | ProofOfModelRelation (ModelRelationIdentity d)
```

Listing A.6: Backend-neutral Lean proof obligation.

The proof module should refuse to render a theorem if the domain cannot express its exact semantic claim in Lean. A solver-only domain can still provide counterexamples and rankings.

Appendix B

# Concrete SMT-LIB encodings

These examples are schematic. Production queries should use the typed internal command IR, canonical renderer, exact symbol-role table, and bounded response schema. Each script was nevertheless run as written against Z3 5.0.0 (each in its own fresh process; see chapter 12 for why that matters), and the answer observed is quoted after the listing.

## B.1 Length-preservation counterexample

Suppose symbolic interpretation derives candidate output length

$$r(n) = \mathrm{ite}(n = 0, 0, n - 1).$$

The contract requires $r(n) = n$ for all $n \in \mathbb{N}$. The counterexample query is:

```
(set-logic QF_LIA)
(declare-const input_0 Int)
(assert (<= 0 input_0))
(define-fun result_len () Int
  (ite (= input_0 0) 0 (- input_0 1)))
(assert (not (= result_len input_0)))
(check-sat)
(get-value (input_0))
```

Listing B.1: Counterexample query for a length-changing candidate.

Z3 5.0.0 answers `sat` with `((input_0 1))`. That model is decoded to natural one and replayed by the exact checked Length problem. The `define-fun` layout shown here is illustrative; the current canonical query plan may inline terms or use a different symbol policy.

## B.2 State-bind wiring counterexample

Use uninterpreted sorts and tuple datatypes:

```
(set-logic ALL)
(declare-sort S 0)
(declare-sort A 0)
(declare-sort B 0)
(declare-datatypes ((Pair 2))
  ((par (X Y) ((mk-pair (fst X) (snd Y))))))
```

```
(declare-fun f (S) (Pair A S))
(declare-fun g (A S) (Pair B S))
(declare-const s0 S)

(define-fun a0 () A (fst (f s0)))
(define-fun s1 () S (snd (f s0)))
(define-fun expected () (Pair B S) (g a0 s1))

; Candidate under test incorrectly calls g a0 s0.
(define-fun actual () (Pair B S) (g a0 s0))
(assert (not (= actual expected)))
(check-sat)
(get-value (s0 s1 actual expected))
```

Listing B.2: Relational state-bind counterexample.

Z3 5.0.0 answers `sat` with a model in which `s0` and `s1` are distinct fresh elements of `S` and `actual`, `expected` are pairs differing in their first component: exactly the separating interpretation the text predicts. A production query should request only values required by its model decoder. Often the domain can replay using equality/disequality observations without decoding the full function interpretation. The response schema must not depend on Z3's arbitrary names for uninterpreted elements.

## B.3 Typed selector sketch

Consider a root with three prepared alternatives on a length sample $n$:

1. return input length $n$;
2. return zero;
3. return $n + 1$.

```
(set-logic QF_LIA)
(declare-const s_root Int)
(assert (and (<= 0 s_root) (< s_root 3)))

(define-fun eval_root ((n Int)) Int
  (ite (= s_root 0) n
  (ite (= s_root 1) 0
                       (+ n 1))))

; CEGIS sample n = 0
(assert (= (eval_root 0) 0))
; CEGIS sample n = 2
(assert (= (eval_root 2) 2))

(check-sat)
(get-value (s_root))
```

Listing B.3: Finite selector sketch on accumulated samples.

Z3 5.0.0 answers `sat` with `((s_root 0))`: alternative zero, the only one consistent with both samples. Djex maps selector zero to the exact prepared graph alternative. No source term is reconstructed from `eval_root`.

## B.4 Assumption-literal core for grammar choices

Suppose a partial search selected alternatives $a_1, a_2, a_3$. Track their consequences:

```
(set-option :produce-unsat-cores true)
(set-option :smt.core.minimize true)
(set-logic QF_LIA)
(declare-const a1 Bool)
(declare-const a2 Bool)
(declare-const a3 Bool)
(declare-const x Int)

(assert (=> a1 (>= x 1)))
(assert (=> a2 (<= x 0)))
(assert (=> a3 (= x 7)))

(check-sat-assuming (a1 a2 a3))
(get-unsat-core)
```

Listing B.4: Core-derived grammar nogood.

Z3 5.0.0 answers `unsat` and returns the core `(a2 a1)`; `a3` is correctly absent. The core yields the learned clause $\neg a_1 \vee \neg a_2$. Note that the solver printed the assumptions in a different order from the one issued; the decoder verifies that every returned symbol is one of the exact assumptions issued for this query and canonicalizes the set before creating a nogood receipt, so print order never reaches the receipt.

## B.5 Optimization of a sketch

```
(set-logic QF_LIA)
(declare-const use_case Bool)
(declare-const use_provider Bool)
(declare-const node_count Int)

; Hard typing and behavioral constraints omitted.
(assert (>= node_count 1))

; Prefer no case split, then no provider, then smaller term.
(minimize (ite use_case 1 0))
(minimize (ite use_provider 1 0))
(minimize node_count)
(check-sat)
(get-model)
(get-objectives)
```

Listing B.5: Lexicographic sketch optimization.

Z3 5.0.0 answers `sat` with `use_case = false`, `use_provider = false`, `node_count = 1` and reports the three objective values `0`, `0`, `1` in declaration order. The production protocol must bind objective order and priority mode. An iterative SAT implementation can instead first solve for `use_case = false`, then provider count, then binary-search or linearly tighten node count.

## B.6 CHC safety query

A relational length abstraction for accumulator reverse can be expressed without element values:

```
(set-logic HORN)
(declare-fun revAccLen (Int Int Int) Bool)

; revAccLen inputLength accumulatorLength resultLength
(assert (forall ((a Int))
  (=> (>= a 0)
      (revAccLen 0 a a))))

(assert (forall ((n Int) (a Int) (r Int))
  (=> (and (> n 0)
           (>= a 0)
           (revAccLen (- n 1) (+ a 1) r))
      (revAccLen n a r))))

; Bad state: result length differs from n + a.
(assert (forall ((n Int) (a Int) (r Int))
  (=> (and (>= n 0) (>= a 0)
           (revAccLen n a r)
           (not (= r (+ n a))))
      false)))

(check-sat)
(get-model)
```

Listing B.6: Schematic CHC relation for accumulator reverse lengths.

In a fresh Z3 5.0.0 process this answers `sat`, and `get-model` returns an interpretation of `revAccLen` equivalent to $n + a - r = 0$, printed as the conjunction $\neg(n + a - r \leq -1) \wedge \neg(n + a - r \geq 1)$: the affine invariant of section 6.5, which the domain would normalize to $r = n + a$ before handing it to Lean. In practice a fixedpoint-specific command plan may use `declare-rel`, `rule`, and `query`. The domain owns translation from any returned invariant to a checked arithmetic predicate before Lean proof generation.

The two front ends report the same fact with opposite words, and the protocol decoder must not let that leak. Under the SMT-LIB HORN encoding above, `sat` means Spacer found a model of all clauses, that is, an inductive invariant for `revAccLen`, so the bad state is unreachable and `get-model` returns the invariant; `unsat` means the clauses derive `false` and a concrete refutation trace exists. Under the legacy fixedpoint front end the answer to `(query Bad)` is `sat` when the query is *derivable*, that is, when the bad state is reachable, and `unsat` when it is not. A fixedpoint protocol must therefore commit to one front end and name outcomes by meaning, "safe with invariant artifact" or "bad state reachable with trace", and never surface a raw `sat`/`unsat` to the layers above.

Appendix C

# Contract grammar and normalization

## C.1 Illustrative abstract grammar

```
Contract      ::= Header Domain Target Requires Ensures
                  ProviderLaws Examples Preferences ProofRequest

Header        ::= format-version Natural
Domain        ::= domain DomainName domain-version Natural DomainOptions
Target        ::= arguments RoleBinding* result RoleBinding
Requires      ::= requires Formula*
Ensures       ::= ensures Formula+
ProviderLaws  ::= provider-laws ProviderLaw*
Examples      ::= examples Example*
Preferences   ::= prefer Preference*
ProofRequest  ::= proof ProofPolicy

Formula       ::= true | false
                | not Formula
                | and Formula+
                | or Formula+
                | implies Formula Formula
                | Relation Term*
                | BoundedForall Binder Range Formula
                | DomainFormula

Term          ::= InputRef | ResultRef | Literal | DomainTerm

ProviderLaw   ::= provider SourceName ProviderRoles
                  ProviderPrecondition ProviderRelation TrustClass

TrustClass    ::= assumed-uniform
                | conditional-on-ground-discharge
                | lean-theorem QualifiedLeanName

ProofPolicy   ::= none | best-effort | lean-required
```

Listing C.1: Illustrative contract grammar, independent of concrete JSON syntax.

## C.2 Normalization requirements

Normalization should be deterministic and domain-owned. Common normalization may:

- validate and canonicalize names without resolving them;
- flatten associative Boolean connectives while preserving source clause identities;
- remove duplicate literal `true`/`false` structure under a versioned rule;
- canonicalize source role order only where order is declared irrelevant;
- bound and normalize integer literal syntax;
- sort sets but preserve lists and objective order;
- alpha-normalize bounded contract binders.

Domain normalization may:

- normalize affine arithmetic and natural monus;
- guard or reject sequence index operations;
- canonicalize bag count sums;
- normalize bit-vector widths and literals;
- expand constructor tag names through exact checked schemas;
- reduce relational state projections;
- normalize CHC variables and rule-local binders.

A fingerprint is computed from the checked normalized form plus exact semantic identities, not raw JSON bytes. Raw bytes may enter acquisition diagnostics or a content digest but carry no semantic authority.

## C.3 Composition

A future contract may combine domains, for example sequence order plus bag multiplicity plus length. Composition should be a checked product:

```
data ProductDomain left right

data ProductContract left right = ProductContract
  { leftContract  :: CheckedDomainContract left
  , rightContract :: CheckedDomainContract right
  , sharedTargetCorrespondence :: SharedTargetReceipt left right
  }
```

Listing C.2: Conceptual checked product of observation domains.

The shared-target receipt proves both domains refer to the same exact target arguments/result and compatible source family. Each subdomain retains its own session, encoding, query and evidence. A combined counterexample may be searched in one merged query only through a product-domain lowering that states its exactness. Otherwise run subqueries and combine replayed results.

Appendix D

# Authority and trust matrix

Table D.1: What each artifact can and cannot justify.

| Artifact | Establishes | Does not establish | Safe search use | Safe user-facing claim |
|---|---|---|---|---|
| Rendered candidate text | A spelling proposed by a renderer | Type correctness, origin, behavior | Submit to Lean callback | "candidate spelling" |
| Callback `Verified` receipt | Exact callback accepted that candidate occurrence | Kernel theorem, behavior, graph availability | Admit to post-verification stages | "accepted by Lean backend callback" |
| Typed candidate graph | Checked graph structure under its sealer | Source inventory membership, provider law, behavior | Interpret in supported domains | "typed graph available" |
| Certificate-associated graph | Exact occurrence/type-application association carried by the atom | Provider behavioral truth or dictionary discharge by itself | Authorize exact provider occurrence after domain checks | "provider occurrence structurally associated" |
| Behavioral problem | Exact domain/inventory/encoding/candidate/problem tuple | Solver result or [illegible] | Key queries and caches | "behavioral problem prepared" |
| Raw `sat` | Solver found a model for query bytes | Valid model decode or source counterexample | Attempt bounded decode | "solver reported sat" |
| Replayed counterexample | Domain evaluator reproduced violation for exact problem/input | Unconditional source runtime theorem beyond model/provider assumptions | Reject in authorized hard mode; add to bank | "contract violated in the stated model" |
| Raw `unsat` | Solver reported no model for query | Lean proof; sound abstraction; complete source semantics | Rank, guide proof, optional trusted mode | "solver reported unsat for this encoding" |

| Artifact | Establishes | Does not establish | Safe search use | Safe user-facing claim |
|---|---|---|---|---|
| Finite exhaustive receipt | Every admitted finite assignment was replayed | Unbounded property outside stated domain | Accept bounded contract; prefer candidate | "validated for stated finite domain" |
| Over-approximate prefix unsat receipt | No completion represented by sound over-approximation satisfies constraints | General impossibility outside grammar/summary | Prune exact prefix; learn scoped nogood | "prefix infeasible under checked completion abstraction" |
| Sketch model | Selector/parameter assignment satisfies current sketch constraints | Lean-valid term or universal behavior | Decode/materialize then verify | "sketch model found" |
| Lean-verified sketch candidate | Materialized candidate elaborates at exact target | Behavioral theorem | Run replay/query/proof | "typed candidate synthesized" |
| Spacer "safe" observation (`sat` in the HORN encoding, `unsat` for a fixedpoint `query`) | CHC bad state not reachable under solver/encoding | Lean induction theorem, termination, exact source semantics | Propose invariant; rank | "CHC solver found no bad state" |
| Lean proof receipt | Lean accepted generated theorem in exact environment | Claims stronger than theorem statement | Stop proof-required search; cache under environment identity | "behavior proved in Lean" |

Appendix E

# Source map of the inspected implementation

## E.1 Leant

The following current files are especially relevant to implementation:

Table E.1: Leant source map at commit `b701ee5`.

| Path | Relevance |
|---|---|
| `README.md` | User-visible `:synth` behavior, engine modes, mandatory Lean verification and current optional Length ranking. |
| `docs/SYNTHESIS_PROPOSAL.md` | Implemented phase history and future synthesis direction. |
| `docs/synth-internals.md` | Exact semantic origins, candidate authority, Length handoff, adapter, ranking and live policy narrative. |
| `src/Leant/Synth/Verification.hs` | Opaque callback-accepted `Verified` receipts and quota-aware variant groups. |
| `src/Leant/Synth/PostVerification.hs` | Rank-2 batch epoch and sealed permutation-only post-verification boundary. |
| `src/Leant/Synth/Engine.hs` | Candidate generation, exact typed origin and semantic authority integration. |
| `src/Leant/Synth/Length/Contract.hs` | Passive scalar/pair Length contracts, exact spine/provider identities, roles and case policy. |
| `src/Leant/Synth/Length/Handoff.hs` | Sole occurrence-sealed conversion from one verified origin to a checked Djex Length problem. |
| `src/Leant/Synth/Length/Integration.hs` | Explicit configuration activation, disabled identity path, command-local contract choice and post-verification assessment. |
| `src/Leant/Synth/Length/Ranking*.hs` | Stable semantic ranking, failures and receipts. |
| `src/Leant/Synth/Length/PostVerification*.hs` | Binding of ranking to exact verified batches and preservation on failures. |
| `src/Leant/Synth/Length/Configuration*.hs` | Startup execution, replay, applicable-domain and launcher policies (schema-versioned at this commit; one versionless schema since `4fa5c0a1`). |

## E.2 Djex

Table E.2: Djex source map at commit `0501e72`.

| Path | Relevance |
|---|---|
| `synthesis/README.md` | Ownership map for the parser/backend-neutral checked synthesis foundation and SMT runtime. |
| `docs/semantic-foundations.md` | Typed candidates, certificates, class resolution, Length semantics, replay and live Z3 specification. |
| `synthesis/src/Language/Haskell/Synthesis/Semantic/Observation.hs` | Honest raw three-valued solver and four-valued behavioral observation vocabulary. |
| `synthesis/src/Language/Haskell/Synthesis/Semantic/Problem.hs` | Generic fingerprinted behavioral problem, associated observations and replay/private evidence seam. |
| `synthesis/src/Language/Haskell/Synthesis/Semantic/Length.hs` | Checked finite-spine contract vocabulary, expressions, formulas, roles and provider summaries. |
| `synthesis/src/Language/Haskell/Synthesis/Semantic/Length/Evaluate.hs` | Bounded deterministic candidate/contract replay and finite-domain validators. |
| `synthesis/src/Language/Haskell/Synthesis/Semantic/Length/Problem.hs` | Atomic session, contract resealing, candidate graph/provider authorization and exact problem identity. |
| `synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib.hs` | Canonical QF-LIA lowering and query-specific replay. |
| `synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/Live.hs` | Scoped capability-probed Z3 worker, query lease, deadlines and heuristic observation handling. |
| `synthesis/src/Language/Haskell/Synthesis/Internal/SMTLib/*` | Typed QF-LIA, response parser, causal stream/driver, lexical policy and direct process runtime. |
| `synthesis/src/Language/Haskell/Synthesis/TypedGenerated*` | Checked term graph, graph fingerprints, certificate plans and association carriers. |
| `exference/*` | Ranked heuristic search and current exact typed-graph producer used by semantic domains. |
| `djinn/*` | LJT search and the remaining typed-graph source-context gap described by the semantic foundation. |

## E.3 Z3

The proposal relies on public solver capabilities documented in the official Z3 guide and source repository:

- incremental scopes and assumptions;
- models and unsat cores;
- arithmetic and soft optimization with ordered objectives;
- algebraic datatypes, arrays, sequences and bit-vectors;
- recursive function definitions for bounded unfolding;
- fixedpoint/Horn-clause engines, especially Spacer;

- the separate Synthesiz3 research artifact as an experimental synthesis direction.

Appendix F

# Implementation review checklist

Before a new domain or synthesis mode is enabled outside tests, reviewers should be able to answer every applicable question below.

## F.1 Contract and source identity

- Is the passive source grammar bounded, and its semantic model identified (by a domain version or an exact domain identifier), before normalization?
- Are all source names resolved against an exact retained inventory/provenance map?
- Are argument/result roles explicit, ordered and arity checked?
- Are hard requirements separate from preferences?
- Are provider laws explicit and classified as assumed, discharged or Lean-proved?
- Does the checked contract fingerprint include every semantic field and exclude irrelevant formatting/allocation fields?

## F.2 Candidate interpretation

- Does the domain consume an exact typed candidate/verified origin rather than rendered text?
- Are graph and schema freshly resealed under domain-owned assumptions?
- Are dead nodes, provider prefixes and certificate occurrences audited where authority depends on them?
- Does every unsupported construct fail closed at an explicit demand point?
- Are opaque values prevented from inspection/application outside admitted operations?
- Is recursion/case/effect policy explicit?

## F.3 SMT lowering and process

- Is the query built from typed syntax and structurally fingerprinted?
- Is the approximation direction exact and nominally represented?
- Are all quantifiers guarded or domain-owned?

- Are sequence indices bounds-guarded and division/shift corner cases explicit?
- Does the protocol request only schema-required artifacts?
- Are executable, version, parameters, seed, launch and response limits part of execution identity?
- Is the protocol state machine specific enough to prevent invalid command sequences?

## F.4 Replay and authority

- Is every decoded model bounded, sorted and checked for missing/duplicate values?
- Does independent replay use the original checked problem, not a simplified surrogate?
- Does evidence bind exact domain/inventory/encoding/candidate/problem/query identities?
- Can a raw `unsat`, core or invariant reach a public evidence constructor? It should not unless explicitly designed and checked.
- Are provider assumptions retained in the receipt and presentation?
- Does a positive claim state finite bounds, model scope or Lean proof status exactly?

## F.5 Search and pruning

- Does hard rejection require a sealed candidate-specific receipt?
- Are unknown and inapplicable policies explicit?
- Is a counterexample bank scoped without storing candidate verdict authority?
- Can semantic deduplication discard an authority-bearing representative? It should retain exact handles.
- Does prefix pruning use only checked over-approximations?
- Are learned nogoods scoped to exact grammar, bank and summary identities?
- Are completeness statements qualified by grammar/search/resource bounds?

## F.6 Lean proof

- Does the proposition renderer correspond to the exact domain semantics?
- Are assumed provider laws hypotheses rather than hidden facts?
- Are candidate definition and theorem checked in the exact active environment?
- Are theorem names/snapshots/restarts handled by normal Leant environment ownership?
- Is proof-search failure distinct from theorem refutation and solver unknown?
- Is only a Lean-accepted receipt labeled “proved in Lean”?

# References

# Bibliography


[1] Leant project. *Leant repository*, commit b701ee59360d6b04456119dc68395ab35326a8ee, August 15, 2026. https://github.com/VladimirReshetnikov/Leant/tree/b701ee59360d6b04456119dc68395ab35326a8ee.

[2] Leant project. *Leant README: REPL and synthesis behavior*, inspected at commit b701ee59360d6b04456119dc68395ab35326a8ee. https://github.com/VladimirReshetnikov/Leant/blob/b701ee59360d6b04456119dc68395ab35326a8ee/README.md.

[3] Leant project. *Proposal: automatic term synthesis in Leant, borrowing from Djex*, inspected at commit b701ee5. https://github.com/VladimirReshetnikov/Leant/blob/b701ee59360d6b04456119dc68395ab35326a8ee/docs/SYNTHESIS_PROPOSAL.md.

[4] Leant project. *Leant synthesis internals*, inspected at commit b701ee5. https://github.com/VladimirReshetnikov/Leant/blob/b701ee59360d6b04456119dc68395ab35326a8ee/docs/synth-internals.md.

[5] Leant project. src/Leant/Synth/Verification.hs, inspected at commit b701ee5. https://github.com/VladimirReshetnikov/Leant/blob/b701ee59360d6b04456119dc68395ab35326a8ee/src/Leant/Synth/Verification.hs.

[6] Leant project. src/Leant/Synth/PostVerification.hs, inspected at commit b701ee5. https://github.com/VladimirReshetnikov/Leant/blob/b701ee59360d6b04456119dc68395ab35326a8ee/src/Leant/Synth/PostVerification.hs.

[7] Leant project. src/Leant/Synth/Length/Contract.hs, inspected at commit b701ee5. https://github.com/VladimirReshetnikov/Leant/blob/b701ee59360d6b04456119dc68395ab35326a8ee/src/Leant/Synth/Length/Contract.hs.

[8] Leant project. src/Leant/Synth/Length/Handoff.hs, inspected at commit b701ee5. https://github.com/VladimirReshetnikov/Leant/blob/b701ee59360d6b04456119dc68395ab35326a8ee/src/Leant/Synth/Length/Handoff.hs.

[9] Leant project. src/Leant/Synth/Length/Integration.hs, inspected at commit b701ee5. https://github.com/VladimirReshetnikov/Leant/blob/b701ee59360d6b04456119dc68395ab35326a8ee/src/Leant/Synth/Length/Integration.hs.

[10] Djex project. *Djex repository*, commit 0501e72edf3fb59f6800072b9ae105d4fdcb972b, August 15, 2026. https://github.com/VladimirReshetnikov/Djex/tree/0501e72edf3fb59f6800072b9ae105d4fdcb972b.

[11] Djex project. *Shared synthesis foundation*, inspected at Djex commit 0501e72. https://github.com/VladimirReshetnikov/Djex/blob/0501e72edf3fb59f6800072b9ae105d4fdcb972b/synthesis/README.md.

[12] Djex project. *Semantic foundations*, inspected at Djex commit 0501e72. https://github.com/VladimirReshetnikov/Djex/blob/0501e72edf3fb59f6800072b9ae105d4fdcb972b/docs/semantic-foundations.md.

[13] Djex project. `Language.Haskell.Synthesis.Semantic.Observation`, inspected at Djex commit `0501e72`. https://github.com/VladimirReshetnikov/Djex/blob/0501e72edf3fb59f6800072b9ae105d4fdcb972b/synthesis/src/Language/Haskell/Synthesis/Semantic/Observation.hs.

[14] Djex project. `Language.Haskell.Synthesis.Semantic.Problem`, inspected at Djex commit `0501e72`. https://github.com/VladimirReshetnikov/Djex/blob/0501e72edf3fb59f6800072b9ae105d4fdcb972b/synthesis/src/Language/Haskell/Synthesis/Semantic/Problem.hs.

[15] Z3 Project. *Z3 repository*, commit `221b77f086b41fac32a761f20e26894f5be1518a`, August 15, 2026. https://github.com/Z3Prover/z3/tree/221b77f086b41fac32a761f20e26894f5be1518a.

[16] Z3 Project. *Z3 5.0.0 release*, July 17, 2026. https://github.com/Z3Prover/z3/releases/tag/z3-5.0.0.

[17] Z3 Project. *Online Z3 Guide: Basic commands and scopes*. https://microsoft.github.io/z3guide/docs/logic/basiccommands/.

[18] Z3 Project. *Online Z3 Guide: Unsat cores and assumptions*. https://microsoft.github.io/z3guide/programming/Z3%20Python%20-%20Readonly/advanced/.

[19] Z3 Project. *Online Z3 Guide: Arithmetical optimization and combining objectives*. https://microsoft.github.io/z3guide/docs/optimization/arithmeticaloptimization/; https://microsoft.github.io/z3guide/docs/optimization/combiningobjectives/.

[20] Z3 Project. *Online Z3 Guide: Algebraic datatypes*. https://microsoft.github.io/z3guide/docs/theories/Datatypes/.

[21] Z3 Project. *Online Z3 Guide: Sequences*. https://microsoft.github.io/z3guide/docs/theories/Sequences/.

[22] Z3 Project. *Online Z3 Guide: Bit-vectors*. https://microsoft.github.io/z3guide/docs/theories/Bitvectors/.

[23] Z3 Project. *Online Z3 Guide: Recursive functions*. https://microsoft.github.io/z3guide/docs/logic/Recursive%20Functions/.

[24] Z3 Project. *Online Z3 Guide: Generalized PDR with Spacer*. https://microsoft.github.io/z3guide/docs/fixedpoints/engineforpdr/.

[25] Petra Hozzová and Nikolaj Bjørner. Synthesiz3 This: an SMT-Based Approach for Synthesis with Uncomputable Symbols. In *FMCAD 2025*, pp. 215–225. https://doi.org/10.34727/2025/isbn.978-3-85448-084-6_28; preprint https://arxiv.org/abs/2504.16536; artifact https://github.com/Z3Prover/doc/tree/master/synthesiz3.

[26] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. Combinatorial sketching for finite programs. In *ASPLOS XII*, 2006. https://doi.org/10.1145/1168857.1168907.

[27] Rajeev Alur, Rastislav Bodik, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *FMCAD*, 2013. https://doi.org/10.1109/FMCAD.2013.6679385.